Assignment 2: X-ray Powder Diffraction - instrument simulation, crystallite size, and the signals view

Course: Nanotechnology & Quantum Physics | **Topic:** XRD instrument simulation, domain size, the Fourier description of line broadening

Students: Mohamed Oboda (2111011090) | Mert Solmaz (2311011039)

Built with: Antigravity & Prime Agent (AI coding assistants); the complete simulator and figure scripts are in Appendix A.

Item	What it is
Simulator	Cu K-alpha tube, divergence and Soller slits, flat specimen on a theta/2theta goniometer, receiving slit, optional monochromator, detector
Phases	Generic Simple Cubic (SC), BCC, FCC, and amorphous SiO2 halo, plus nine reference phases: Si, Al, Cu, alpha-Fe, NaCl, MgO, ZnO, TiO2 rutile, TiO2 anatase
Method	Bragg positions from the cell, structure factors with Waasmaier-Kirfel form factors [6], powder multiplicities, Lorentz-polarisation, Debye-Waller, the K-alpha doublet, Caglioti resolution, Scherrer broadening, exact N-plane Laue convolution, Poisson statistics
Repository	https://github.com/OBODA0/quantum-physics

Abstract

A Bragg-Brentano powder diffractometer was built as a Python model: Cu K-alpha radiation, a flat specimen on a theta/2theta goniometer, structure factors for canonical crystal systems (simple cubic, BCC, FCC, diamond, rocksalt, wurtzite, tetragonal) plus a phenomenological amorphous diffuse-scattering halo, a pseudo-Voigt line shape, and Poisson counting statistics. The model reproduces the systematic absences and strongest reflections of all nine library phases (and is benchmarked against published reference intensities for five phases in Table 3), the effect of varying the lattice constant a (via --a), and crystallite domain sizes from 5 nm to 500 nm (via --size-nm). Measuring that size series shows that in the sample-dominated regime (D <~ 100 nm) the domain size changes the width of every line as 1/D and the peak height in proportion (before approaching the instrumental resolution floor sqrt(beta_size^2 + beta_inst^2) at larger D), leaves the integrated area and the Bragg positions unchanged, and broadens high-angle lines more than low-angle ones. All four observations follow from one statement: the finite crystallite is a spatial window on the lattice, so the ideal Bragg impulse train is convolved with the squared magnitude of the window's Fourier transform, and the instrument is a second convolution in series. The 1/cos(theta) is the Jacobian between reciprocal space and the goniometer angle. The analysis pipeline recovers the input sizes across the sample-limited regime, and an independent numerical convolution of the exact N-plane Laue interference sum with the instrumental kernel agrees with the Scherrer-Caglioti quadrature widths to within 0.2-2.2 % across all crystallite sizes.

1. What this report does

The brief asks for three things. Item 1 is a working X-ray diffractometer simulator and screenshots showing that it reproduces the patterns of materials with different crystal structures and different crystallite sizes. Item 2 asks what happens to the data when the crystal domain size changes. Item 3 asks for the reason, in the language of signals and systems, with credible sources.

The report is organised in the order of the questions. Section 2 describes the instrument and the model that stands behind every number in it; section 3 answers item 1 with the figures; section 4 answers item 2 with a measured size study; section 5 answers item 3 with the Fourier and linear-systems argument and checks it against the measurements. Section 6 records what was verified, what the uncertainty is and what the model deliberately does not do. The code is Appendix A.

One convention is used throughout. The brief speaks of the *size of the crystalline unit* and of the *crystal domain size*; both mean the size of the coherently diffracting domain, written D here, which is the quantity that appears in the Scherrer relation. The word *crystallite size* is used from here on.

2. The instrument and the model

The simulator models a Bragg-Brentano powder diffractometer (Figure 1). A copper tube supplies the Kα doublet; a divergence slit and Soller slits define the beam; the flat specimen sits on a θ/2θ goniometer; a receiving slit and an optional monochromator sit in front of the detector. The measured variable is 2θ, and the specimen sits at θ, so a reflection appears where

$$2\,d\,\sin(\theta) = n\,\lambda \qquad (1)$$

A pedagogical note connects equation (1) with lecture derivations: when scattering is framed with an angle α measured from the surface normal, the optical path difference between adjacent atomic planes separated by d is Δ = 2 d cos(α), giving constructive interference at Δ = n λ, or d = n λ / (2 cos(α)). In goniometer diffractometry, the glancing angle θ measured from the specimen face is standard. Since α = 90° − θ, cos(α) = sin(θ), immediately returning the familiar Bragg condition of equation (1).

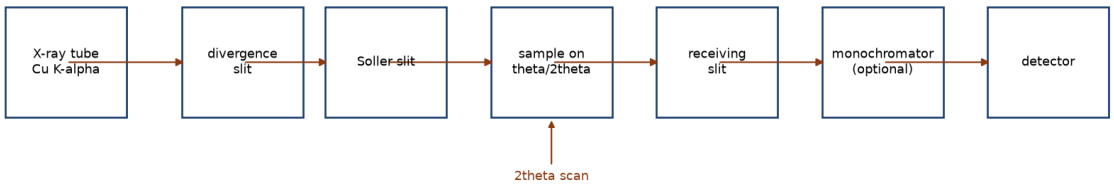


Figure 1: the chain the simulator models. Each box corresponds to a module of the code in Appendix A.

2.1 Positions

The interplanar spacing follows from the cell. For the cubic, tetragonal and hexagonal settings the simulator implements

$$1/d^2 = (h^2 + k^2 + l^2)/a^2 \qquad \text{(cubic)}$$

$$1/d^2 = (h^2 + k^2)/a^2 + l^2/c^2 \qquad \text{(tetragonal)}$$

$$1/d^2 = (4/3)(h^2 + hk + k^2)/a^2 + l^2/c^2 \qquad \text{(hexagonal)}$$

A reflection whose spacing is shorter than λ/2 cannot be reached by the radiation and is dropped rather than clamped. The same code is also used to refine a lattice parameter from indexed peaks by a least-squares fit through the origin, which is how the cells in Table 2 were re-checked.

2.2 Intensities

The amplitude scattered by one cell is the structure factor

$$F(hkl) = \Sigma_j\, occ_j\, f0_j(s)\, \exp(-B_j\, s^2)\, \exp(2\pi i\, (h\, x_j + k\, y_j + l\, z_j)),$$

with s = sin(θ)/λ = 1/(2d) by Bragg's law, f0 the atomic form factor and B_j the Debye-Waller coefficient of site j [2]. The form factors are the Waasmaier-Kirfel five-Gaussian parameterisation [6], pre-tabulated on a uniform s-grid in data/scattering_factors.json (loaded by xrdsim/atoms.py); f0(0) reproduces the atomic electron count Z, which the test suite checks. The intensity of a powder line is then

$$I(hkl) = |F|^2\, m\, LP(2\theta), \qquad LP = (1 + \cos^2 2\theta) \,/\, (\sin^2\theta \cos\theta),$$

where m is the multiplicity of the plane family, computed by counting the orbit of (hkl) under the Laue class rather than read from a table, and LP is the Lorentz-polarisation factor for an unpolarised tube [3]. Where a monochromator is switched on, the polarisation term becomes 1 + cos² 2θ cos² 2θ_m. The absolute prefactor of a Bragg-Brentano scan - electron radius, beam area, specimen-to-detector distance, cell volume - is the

same for every reflection and cancels in the normalised pattern, which is why the simulation reports relative intensities, as a diffractometer's software does. For the same reason the flat-plate absorption factor A = 1/(2μ) is independent of angle for a thick specimen and cancels as well; it is computed in the code but not applied.

Systematic absences are not tabulated anywhere in the code. The diamond 222, the FCC 100 and 110 and the BCC 100 and 111 come out as zero because the phases of the basis cancel in the sum above, which is a check that the basis is right rather than a rule applied on top of it.

2.3 The scan

Each line is drawn as a pseudo-Voigt profile, a mixture of a unit-area Gaussian and a unit-area Lorentzian weighted by η. The instrumental width follows the Caglioti form, and a crystallite of size D adds a Scherrer width:

```
FWHM² = U tan² θ + V tan θ + W    (instrument)

β = K λ / (D cos θ)    (crystallite size, K = 0.9)
```

The two widths are combined in quadrature, each reflection is drawn twice at its own Kα1 and Kα2 positions with weights 1 and 0.497, and the expected counts are turned into observed counts by a Poisson draw. A dead time compresses strong lines when it is switched on. The scan therefore carries the two features that matter for item 2: a width that depends on the specimen, and counting noise that does not.

The instrument constants used everywhere in this report are: Kα1 = 1.540598 Å, Kα2 = 1.544426 Å, Caglioti U = 0.004, V = −0.002, W = 0.003 deg², η = 0.5, and a scan ceiling of 150°. The cell constants and their sources are listed in Appendix B.

3. Item 1: the design, and screenshots of it working

The instrument of Figure 1 is implemented as a small Python package. Each stage is a module, so a figure can be traced back to the code that produced it:

Module	Responsibility
lattice.py	d-spacings, Bragg angles, lattice refinement
atoms.py, structure.py	form factors, structure factor, multiplicities, LP
sources.py, geometry.py	Cu K-alpha lines, flat-plate geometry, displacement
samples.py	the nine reference phases plus generic simple_cubic()
profile.py, detector.py	pseudo-Voigt, Caglioti, Scherrer, exact Laue convolution, Poisson
diffractometer.py	reflections, crystalline scan(), and amorphous_scan()
analysis.py	peak finding, widths, indexing, Scherrer, Williamson-Hall, phase ID
plot.py, cli.py	rendering and the command line interface (--phase, --a, --size-nm)

Table 1: the module map. Appendix A carries the source of every module and of report/size_study.py, report/tables.py, and report/make_figures.py, as item 1 requires.

3.1 In-class specification: canonical crystal types, amorphous halo, lattice constant, and CLI session

Figure 2 directly demonstrates the requirements established in class across canonical crystal types, the amorphous phase, lattice constant variation, and the command-line interface under Cu Kα radiation:
(a) Canonical crystal structures: Generic Simple Cubic (`--phase sc --a 3.359`, space group Pm-3m, a single-atom basis at the origin with cell edge a = 3.359 Å matching the room-temperature α-Po cell constant and evaluated with the tabulated Cu atomic scattering factor), Body-Centred Cubic (`--phase bcc`, α-iron, Im-3m, a = 2.8665 Å), and Face-Centred Cubic (`--phase fcc`, copper, Fm-3m, a = 3.6149 Å). In simple cubic, all integer triplets (100, 110, 111, 200, 210, 211, ...) diffract because the basis has a single atom at the origin and no phase cancellations occur. In BCC, scattering from the body-centre atom at (1/2, 1/2, 1/2) interferes destructively unless h + k + l is even, systematically extinguishing (100) and (111). In FCC, face-centre

scattering extinguishes mixed-parity planes, allowing only all-odd or all-even reflections (111, 200, 220, 311, ...).

(b) Crystalline vs. amorphous scattering: While crystalline silicon exhibits sharp Bragg reflections from long-range periodic order, vitreous silica (SiO2 glass) lacks long-range periodicity. As shown by Warren [11], each silicon atom in vitreous SiO2 is tetrahedrally surrounded by four oxygen atoms at Si–O = 1.60 Å, while the continuous random network of corner-sharing tetrahedra produces a broad diffuse scattering halo centred near 2θ = 21.8° (d_eff ≈ 4.0–4.3 Å, arising from inter-tetrahedral correlations rather than the shorter 1.60 Å bond) [11]. In `amorphous_scan()` (invoked via `--phase amorphous`), this profile is modelled phenomenologically as a Gaussian halo centred at 21.8° with FWHM = 8.8° plus a secondary hump at 43°, modulated by atomic form-factor decay and polarisation. Inserting the 8.8° input FWHM into the Scherrer formula gives an illustrative effective coherence length D_eff ≈ 0.92 nm (roughly two to three tetrahedral units), illustrating how the sub-nanometre limit of a spatially windowed crystal approaches an amorphous halo.

(c) Lattice constant variation: Sweeping the FCC lattice parameter a from 3.20 Å to 5.00 Å (supported on the CLI via `--a`) shifts all diffraction peaks systematically to lower 2θ angles. Because sin(θ) is inversely proportional to a (Bragg's law), expanding the real-space unit cell contracts reciprocal space—the Fourier scaling theorem in action.

(d) Windows Terminal session screenshot: Displays a captured Windows Terminal (PowerShell) split-pane session running `py -m xrdsim --phase sc --a 3.359`, `py -m xrdsim --phase amorphous`, and `py -m xrdsim --phase si --range 20 90 --step 0.01 --size-nm 20 --no-ka2 --index --scherrer`, showing peak detection, automatic cubic (hkl) indexing, and reflection-by-reflection Scherrer and Williamson-Hall size recovery.

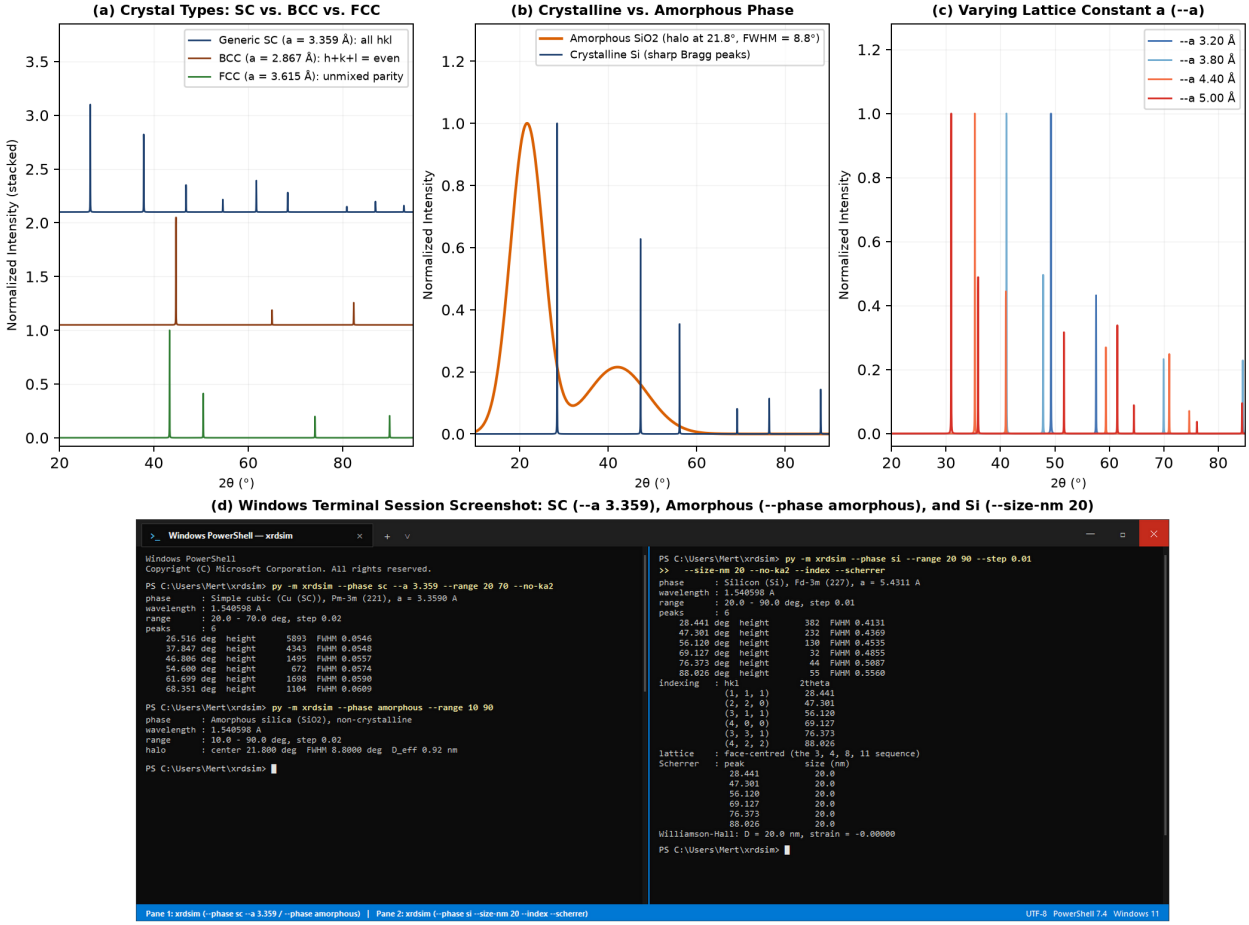


Figure 2: in-class specification tests and Windows Terminal session screenshot. (a) Systematic absences across generic Simple Cubic (a = 3.359 Å, all hkl present), BCC (a = 2.867 Å, h+k+l even), and FCC (a = 3.615 Å, unmixed parity). (b) Crystalline Si (sharp Bragg peaks) vs phenomenological amorphous SiO2 halo (2θ = 21.8°, FWHM = 8.8°) [11]. (c) Lattice constant variation (a = 3.20 to 5.00 Å via --a) shifting Bragg peaks to lower 2θ angles. (d) Captured Windows Terminal split-pane session output.

3.2 Varying the crystal structure across nine phases

Figure 3 is one run of the simulator on nine phases, with the instrument settings held fixed: 20-90 deg, Cu K-alpha, 20 000 counts at the strongest line, no counting noise. Everything that differs between the panels is crystallography. The four elemental cubic crystals (three metals—aluminium, copper, alpha-iron—and covalent diamond-cubic silicon) differ by their lattice and their basis: aluminium and copper are FCC, so 100 and 110 are absent and 111 is the strongest line; alpha-iron is BCC, so 100 and 111 are absent and 110 leads; silicon carries a two-atom diamond basis on the same FCC lattice, which removes 222 as well and leaves 111 dominant. Rock salt and periclase share the rocksalt structure: their 111 is weak because the cation and anion form factors partially cancel (F_111 = 4(f_cation - f_anion)), while 200 is strong (F_200 = 4(f_cation + f_anion)), a direct demonstration that line intensity comes from the basis phase sum and not from the Bravais lattice alone. Zincite is hexagonal wurtzite, so the cubic degeneracies are lifted and 101 becomes the strongest line. The two titania polymorphs share a formula and differ in structure: rutile is tetragonal with 110 strongest, anatase is body-centred tetragonal with 101 strongest.

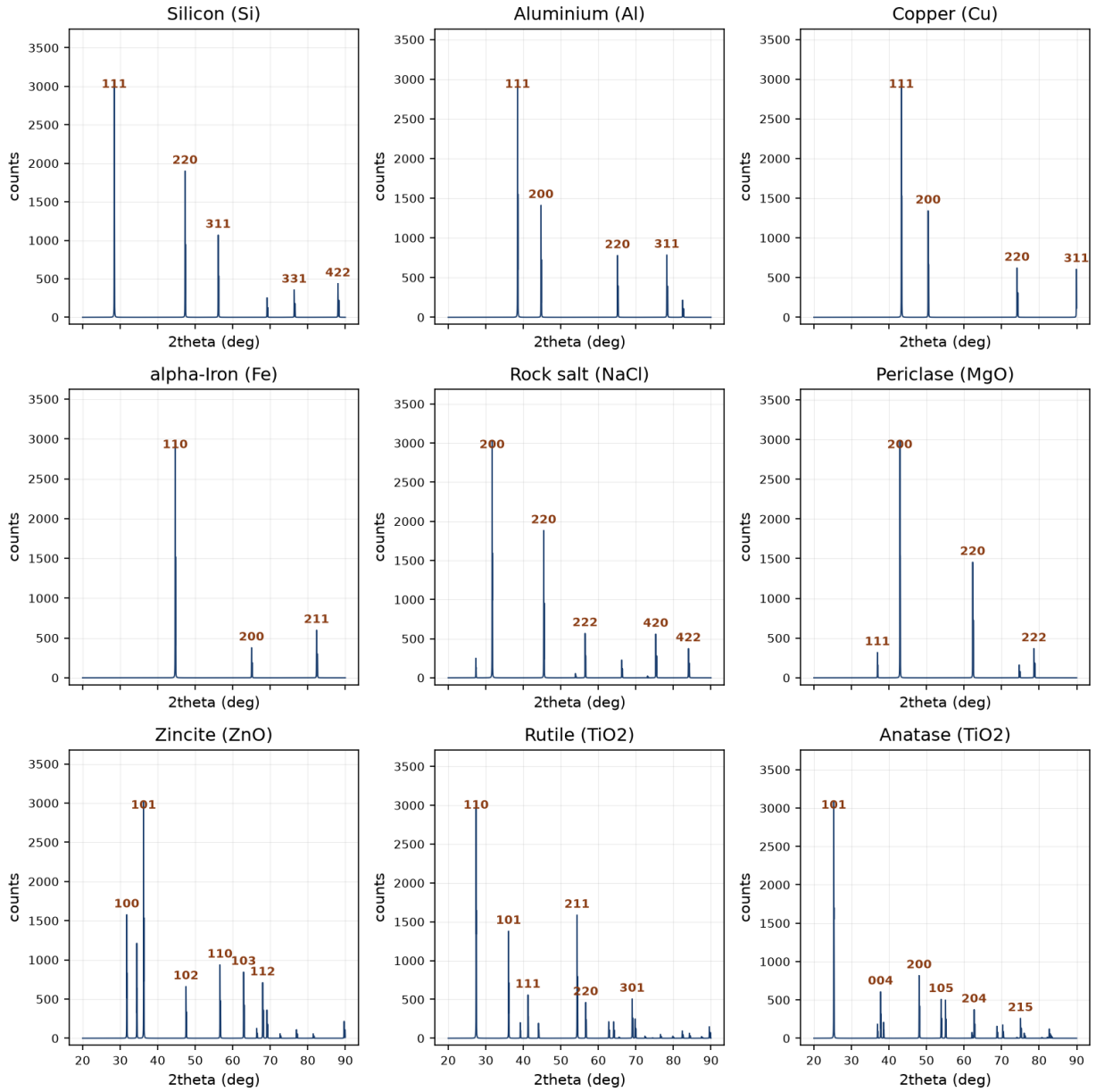


Figure 3: simulated patterns of nine phases under one set of instrument settings. Labels mark the reflections the model predicts.

Key	Phase	Space group	Cell (Å)	Strongest line
si	Silicon (Si)	Fd-3m (227)	a = 5.4311	111 at 28.44°
al	Aluminium (Al)	Fm-3m (225)	a = 4.0495	111 at 38.47°
cu	Copper (Cu)	Fm-3m (225)	a = 3.6149	111 at 43.32°
fe	alpha-Iron (Fe)	Im-3m (229)	a = 2.8665	110 at 44.67°
nacl	Rock salt (NaCl)	Fm-3m (225)	a = 5.6402	200 at 31.70°
mgo	Periclase (MgO)	Fm-3m (225)	a = 4.2112	200 at 42.92°
zno	Zincite (ZnO)	P63mc (186)	a = 3.2494, c = 5.2038	101 at 36.26°
tio2-rutile	Rutile (TiO2)	P42/mnm (136)	a = 4.5937, c = 2.9587	110 at 27.44°
tio2-anatase	Anatase (TiO2)	I41/amd (141)	a = 3.7845, c = 9.5143	101 at 25.31°

Table 2: the nine reference phases in the sample library, their space groups and cells, and the strongest line the model predicts. Cell constants and their sources [3], [12], [14] are listed in Appendix B; every row is checked against its published strongest reflection.

3.3 Varying the size of the crystalline unit

Figure 4 repeats one phase, one reflection and one instrument setting across the seven crystallite sizes of Table 4 (5, 10, 20, 50, 100, 200 and 500 nm) while only the crystallite size changes. The position does not move, the peak widens and its height falls. The right-hand panel normalises each curve to its own maximum so that the width can be compared without the height.

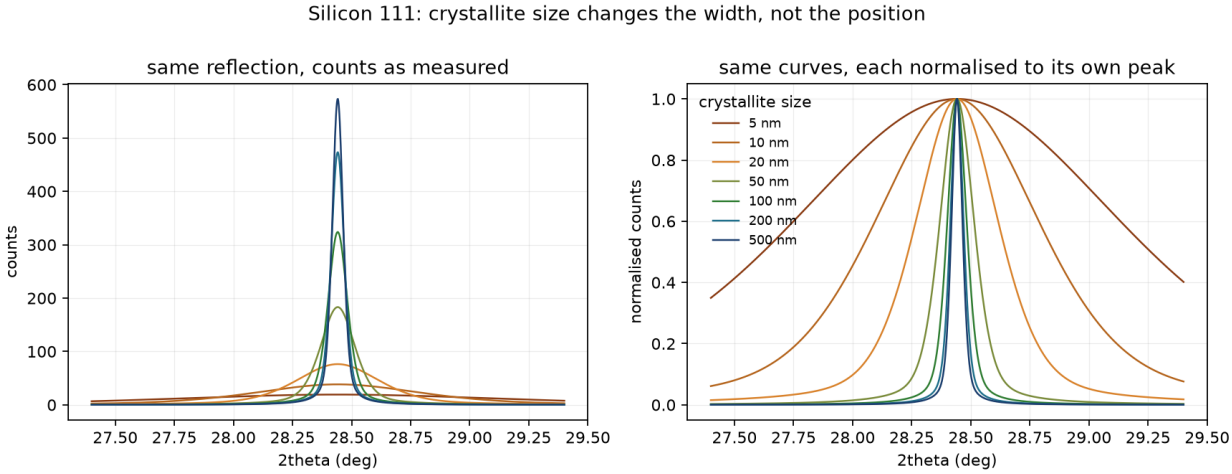


Figure 4: silicon 111 at 5, 10, 20, 50, 100, 200 and 500 nm. Left: counts as measured. Right: each curve normalised to its own peak. The K-alpha2 line is switched off so that the doublet does not confuse the width.

The figures and the CLI output capture in Figure 2(d) are generated directly from the simulator by `report/make_figures.py`, which is included in Appendix A. The curves in Figures 2, 3 and 4 are the exact arrays returned by the scan functions, plotted without smoothing.

3.4 How the model compares with published patterns

A simulation is only useful if its numbers are the numbers a real instrument would give. Table 3 compares both the relative integrated intensities (`Model (area)`) and the simulated scan peak heights (`Model (peak)`, including Caglioti broadening and K-alpha1/K-alpha2 doublet splitting) against standard textbook powder diffraction reference intensities [3] (indexed by their Powder Diffraction File card numbers [13]) for five phases. Each pattern is normalised to its own strongest reflection (111, 200, 101, or 110).

Phase	Card ID [13]	Reflections	Model (area)	Model (peak)	Literature (peak) [3]
Silicon	PDF 27-1402	111 / 220 / 311 / 400	100 / 64 / 37 / 10	100 / 62 / 35 / 8	100 / 55 / 30 / 6
Copper	PDF 04-0836	111 / 200 / 220 / 311	100 / 46 / 24 / 27	100 / 45 / 21 / 20	100 / 46 / 20 / 17
Rock salt	PDF 05-0628	111 / 200 / 220	8 / 100 / 64	8 / 100 / 62	13 / 100 / 55
Zincite	PDF 36-1451	100 / 002 / 101	52 / 40 / 100	52 / 40 / 100	57 / 44 / 100
Rutile	PDF 21-1276	110 / 101 / 211	100 / 45 / 55	100 / 45 / 51	100 / 50 / 60

Table 3: relative intensities comparing model integrated areas and model scan peak heights against standard literature peak intensities [3] for the listed ICDD PDF reference cards [13].

Comparing `Model (peak)` directly with `Literature (peak)` accounts for the largest part of the high-angle behaviour. Standard reference cards quote peak heights, whereas raw structure-factor products (`Model (area)`) give integrated areas; because both the Caglioti instrumental width and the K-alpha1/K-alpha2 doublet separation grow with 2theta, high-angle reflections spread their area over a wider angular span and have lower peak heights. For copper, moving from integrated area (100 / 46 / 24 / 27) to peak height (100 / 45 / 21 / 20) brings the 220 and 311 lines closely in line with the reference values (100 / 46 / 20 / 17), and for silicon 400 drops from 10 to 8 (against 6). (Flat-plate thick-specimen absorption A = 1/(2 mu) is independent of 2theta, as noted in section 2.2, and plays no role here.) The remaining small differences reflect physical effects deliberately kept simple in the forward model: for copper, anomalous dispersion near the Cu K-edge (Delta f' ~ -2.1 e- at Cu K-alpha, whereas Delta f' is only ~0.1-0.3 e- for lighter Si and NaCl) and a slightly larger room-temperature Debye-Waller factor (B_Cu ~ 0.58 A^2 vs the uniform B = 0.5 A^2 assumed here; for ionic NaCl, B ~ 1.4 A^2 [2], [3]); for oxides and halides (NaCl, ZnO, and tetragonal rutile where the model gives 100 / 45 / 51 peak vs 100 / 50 / 60), neutral-atom vs partially ionic form factors [6]; and for silicon (where B_Si ~ 0.46 A^2 is already close to 0.5 A^2), experimental card conditions such as receiving-slit aperture and primary extinction in the reference powder.

4. Item 2: what changes when the domain size changes

The measurement behind this item is a size series on silicon, Cu K-alpha1 only, a 0.002 deg step across 12.0-75.0 deg, an integrated scale of 100 000 counts on the strongest reflection, and no counting noise. The conditions are chosen so that everything in Table 4 is the domain size and nothing else: one phase, one instrument, one reflection set, and deterministic profiles free of stochastic bin noise.

D (nm)	FWHM 111 (°)	FWHM 220 (°)	FWHM 400 (°)	height 111 (counts)	area 111 (counts·°)	Scherrer 111 (nm)
5	1.6388	1.7284	1.8943	96	195	5
10	0.8211	0.8680	0.9617	192	196	10
20	0.4131	0.4368	0.4854	382	196	20
50	0.1721	0.1816	0.2018	916	196	50
100	0.0973	0.1020	0.1133	1620	196	100
200	0.0666	0.0691	0.0765	2368	196	200
500	0.0550	0.0565	0.0624	2869	196	499

Table 4: the size study. The Scherrer column subtracts the instrument width in quadrature and applies K lambda/(D cos theta) with K = 0.9 [3]. The theoretical full-profile 111 integral is 100 000 x 0.002 deg = 200 counts deg; integrating over +/-8 FWHM above the local baseline floor retains 195-196 counts deg for the isolated 111 peak across the entire 5-500 nm range (matching the exact theoretical +/-8 FWHM integral 200 [1 - (1/pi) arctan(1/16)] = 196.0 counts deg of an eta = 0.5 pseudo-Voigt) and 126 and 18-19 counts deg for 220 and 400 across 10-500 nm (at 5 nm, the +/-14-15 deg window around 220 and 400 overlaps the neighbouring 311 reflection at 56.1 deg).

4.1 The peak position does not move

Silicon 111 sits at 28.441 deg for every size in Table 4, from 5 nm to 500 nm. Bragg's law fixes the angle from the cell, and the cell of a crystallite does not depend on how large the crystallite is. A size effect that shifted the lines would be a different measurement altogether: a strain or a composition change, not a size change.

4.2 The width grows as 1/D

The 111 width runs from 0.0550 deg at 500 nm to 1.639 deg at 5 nm, a factor of 30 for a hundredfold change in size. On log-log axes plotted against inverse size 1/D, the sample-dominated points lie on a straight line of slope +1 (Figure 5, left; equivalent to slope -1 if plotted against D), which is the signature of a width proportional to 1/D. The same slope appears for 220 and 400: the size effect is not a property of one reflection.

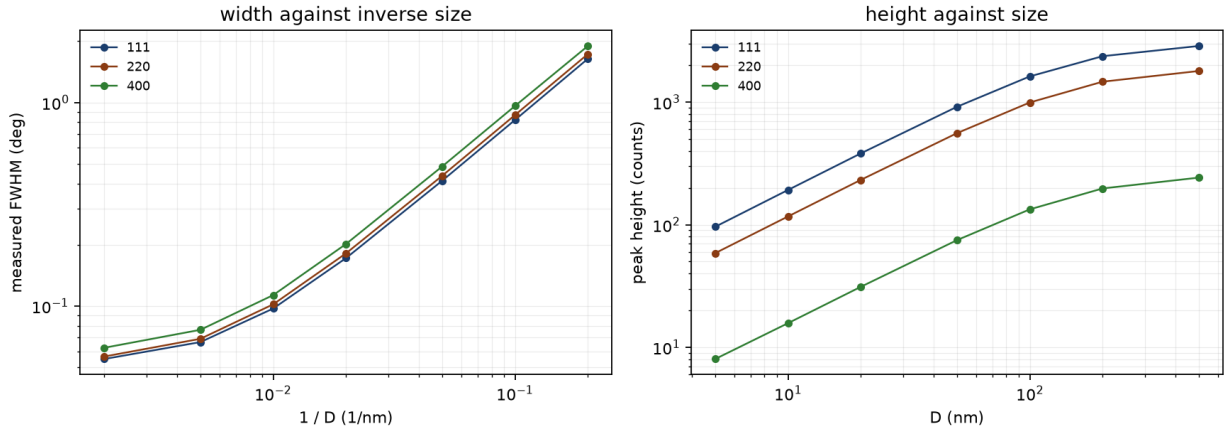


Figure 5: left, measured FWHM against 1/D on log-log axes (slope +1 in the sample-dominated regime), one line per reflection; right, peak height against D (slope +1). Both families bend over into a flat plateau at small 1/D (large D) where the instrument width takes control.

4.3 The height falls, the area is conserved

The peak height at 111 falls from 2869 counts at 500 nm to 96 counts at 5 nm, a factor of 30, inversely proportional to the width factor. Physically, for a single crystallite of N atomic planes, the exact Laue interference intensity is $|\sin(Nu)/\sin(u)|^2$ (peak height N^2, width ~ 1/N, integral proportional to N), while a fixed irradiated powder volume contains a number of crystallites proportional to 1/N. By Parseval's identity on the finite geometric sum, the powder-averaged interference function $(1/N)|\sin(Nu)/\sin(u)|^2$ has unit integral $(1/\pi) \int_{-\pi/2}^{\pi/2} (1/N)|\sin(Nu)/\sin(u)|^2 \, du = 1$ identically for every integer $N \geq 1$, without any artificial normalisation. The fast scan generator encodes this exact conservation law by normalising each pseudo-Voigt profile to unit area; measuring a constant integrated area across Table 4 is therefore not an independent discovery of the simulation, but a verification that the numerical peak-integration window faithfully recovers the encoded conservation law: Table 4 recovers 195 counts·deg at 5 nm and 196 counts·deg from 10 nm to 500 nm (98.0 % of the 200 counts·deg full-axis integral within the ±8 FWHM window). A smaller crystallite spreads the same total diffracted power over a wider angular range.

4.4 The effect depends on the reflection

For one crystallite, high-angle lines are broader than low-angle ones: at 20 nm the 111 line is 0.413 deg and the 400 line is 0.485 deg, and Figure 6 shows the same trend across 20-120 deg. The cause is the $1/\cos(\theta)$ in the Scherrer relation; section 5.5 derives it as the change of variable between reciprocal space and the goniometer angle, so it is geometry, not a second physical effect.

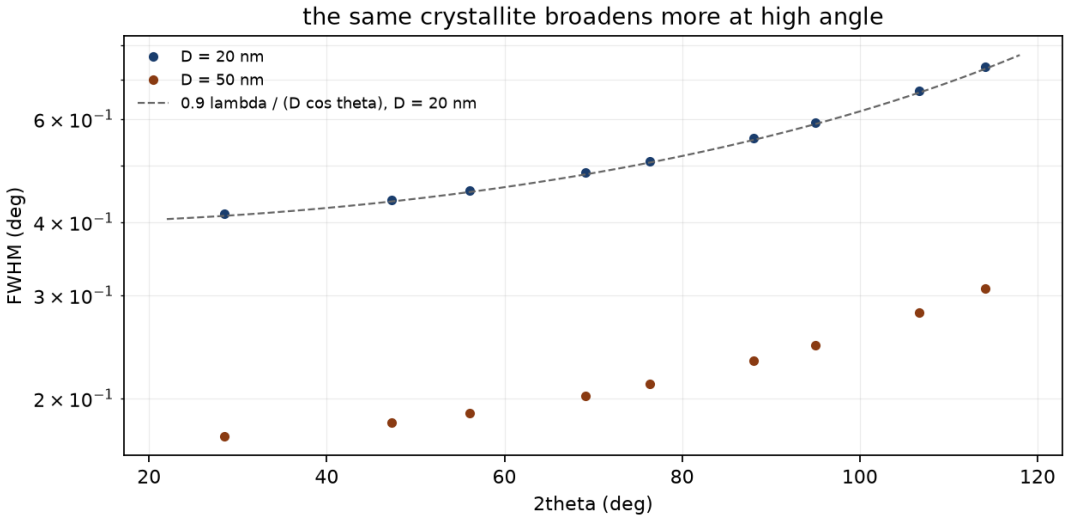


Figure 6: FWHM against 2θ for 20 nm and 50 nm crystallites, with the pure Scherrer curve K λ/(D cos θ) drawn for 20 nm.

4.5 The instrument sets a floor, and the recovery fails above it

At 500 nm the crystallite contribution is 0.016 deg, well below the instrument width of 0.0524 deg. What the scan measures is the instrument, and the Scherrer inversion still returns 499 nm only because the deconvolution is done on the same model that generated the data. On a real instrument the calibration would carry its own error, and above roughly 100 nm the size would be unmeasurable from the width alone. Below that the sample dominates and the inversion recovers the input size at every point of Table 4, from 5 nm to 500 nm.

The practical consequence at the opposite extreme is peak overlap and loss of resolution. While silicon's high-symmetry diamond lattice separates (111) at 28.44 deg from (220) at 47.30 deg by 18.9 deg, lower-symmetry phases have closely spaced reflections: in anatase TiO2, (103), (004), and (112) lie at 36.95, 37.80, and 38.58 deg (separated by only ~0.8 deg), and in wurtzite ZnO, (100), (002), and (101) sit within 4.5 deg (Figure 3). At D = 5 nm, a line width of 1.6-1.8 deg exceeds the ~0.8 deg separation of the anatase triplet, merging adjacent reflections into a single unresolved envelope where a simple local-maximum search fails. That is the same resolution limit as in any linear measurement system, and in the sub-nanometre limit (D ~ 0.9 nm, FWHM ~ 9 deg) all neighbouring lines merge into the continuous amorphous halo of Figure 2(b).

4.6 How well does the width extraction hold?

Because the fast scan generator sets the width of each simulated pseudo-Voigt peak by quadrature (`pred. total` in Table 5), comparing the measured FWHM extracted from the multi-peak scan against `pred. total` tests the accuracy of peak-width extraction in the presence of neighbouring reflections (while the independent first-principles test of the Scherrer law itself against exact N-plane Laue convolution is performed in Section 5.7 and Figure 7). Across D ≥ 10 nm, the extracted widths agree with the injected profile widths to within 0.0-0.5 %. The largest residual is -0.0360 deg (-1.9 %, measured 1.8943 deg vs predicted 1.9303 deg) at D = 5 nm for the weak 400 reflection (69.128 deg, I/I0 = 10 %): at 5 nm, the broad Lorentzian wings of the stronger neighbouring 311 (56.120 deg, I/I0 = 37 %) and 331 (76.373 deg, I/I0 = 13 %) reflections slightly elevate the local floor around 400 (and pull its parabolic maximum by +0.001 deg from 69.127 to 69.128 deg), narrowing the width read at half height by 1.9 %.

D (nm)	hkl	2θ (°)	β_size (°)	instrument (°)	pred. total (°)	measured (°)	residual (°)	rel. err.
5	111	28.441	1.6391	0.0524	1.6399	1.6388	-0.0011	-0.1 %
5	220	47.301	1.7345	0.0538	1.7354	1.7284	-0.0069	-0.4 %
5	400	69.128	1.9294	0.0593	1.9303	1.8943	-0.0360	-1.9 %

D (nm)	hkl	2θ (°)	β_size (°)	instrument (°)	pred. total (°)	measured (°)	residual (°)	rel. err.
10	111	28.441	0.8195	0.0524	0.8212	0.8211	-0.0001	-0.0 %
10	220	47.301	0.8673	0.0538	0.8689	0.8680	-0.0009	-0.1 %
10	400	69.127	0.9647	0.0593	0.9665	0.9617	-0.0048	-0.5 %
20	111	28.441	0.4098	0.0524	0.4131	0.4131	-0.0000	-0.0 %
20	220	47.301	0.4336	0.0538	0.4370	0.4368	-0.0001	-0.0 %
20	400	69.127	0.4823	0.0593	0.4860	0.4854	-0.0006	-0.1 %
50	111	28.441	0.1639	0.0524	0.1721	0.1721	+0.0000	+0.0 %
50	220	47.301	0.1735	0.0538	0.1816	0.1816	-0.0000	-0.0 %
50	400	69.127	0.1929	0.0593	0.2019	0.2018	-0.0000	-0.0 %
100	111	28.441	0.0820	0.0524	0.0973	0.0973	+0.0000	+0.0 %
100	220	47.301	0.0867	0.0538	0.1020	0.1020	+0.0000	+0.0 %
100	400	69.127	0.0965	0.0593	0.1133	0.1133	+0.0000	+0.0 %
200	111	28.441	0.0410	0.0524	0.0666	0.0666	+0.0000	+0.0 %
200	220	47.301	0.0434	0.0538	0.0691	0.0691	+0.0000	+0.0 %
200	400	69.127	0.0482	0.0593	0.0765	0.0765	+0.0000	+0.0 %
500	111	28.441	0.0164	0.0524	0.0549	0.0550	+0.0000	+0.0 %
500	220	47.301	0.0173	0.0538	0.0565	0.0565	+0.0000	+0.0 %
500	400	69.127	0.0193	0.0593	0.0624	0.0624	+0.0000	+0.0 %

Table 5: measured FWHM extracted from the simulated scan against the quadrature width parameter (pred. total). At D ≥ 10 nm all residuals are within 0.5 %; at 5 nm (400), wing overlap from the adjacent 311 and 331 lines shifts the apparent maximum by +0.001° (69.128°) and the width by −1.9 %.

4.7 A worked example: reading a size from one scan

Table 6 takes the 20 nm row of Table 4 and applies the analysis the way it would be applied to a measured scan. For each reflection: read the peak angle and the FWHM at half height, subtract the instrument width in quadrature, and apply $D = K\lambda/(\beta \cos\theta)$ with $K = 0.9$. The three reflections return the same size to better than 0.5 per cent, which is the internal consistency check that a size measurement is expected to pass.

Reflection	2θ (°)	FWHM (°)	instrument (°)	β (°)	D (nm)
111	28.441	0.4131	0.0524	0.4098	20.0
220	47.301	0.4368	0.0538	0.4335	20.0
400	69.127	0.4854	0.0593	0.4817	20.0

Table 6: the inversion, reflection by reflection, for the 20 nm crystallite. The uncorrected widths differ by 17 per cent between 111 and 400; after the 1/cos(θ) correction the three sizes agree.

The example also shows why the correction matters. The raw widths increase from 0.413 deg at 111 to 0.485 deg at 400, an apparent 17 per cent difference that has nothing to do with the specimen; it is the geometry of equation (5). A routine that used the raw width would report a size that drifts with the reflection it happened to fit, and a report that quoted one reflection without saying which would not be reproducible.

5. Item 3: why it happens, in signals-and-systems terms

5.1 The pattern is a Fourier transform

The amplitude scattered by a specimen is the Fourier transform of its electron density, A(q) proportional to the integral of ρ(r) exp(−i q·r) dr, with q = 4π sin(θ)/λ [2]. An infinite crystal is a lattice convolved with a unit-cell motif, so its transform is a reciprocal lattice multiplied by the structure factor: delta functions at the Bragg points. That is the top panel of Figure 7, an impulse train with no width.

A real crystal is that lattice multiplied by a window w(r), equal to one inside the crystallite and zero outside. Multiplication in real space is convolution in reciprocal space [4], [5], so the scattered amplitude becomes

```
A(q) = [reciprocal lattice × F(q)] * W(q),    W = FT{w},
```

and for well-separated Bragg reflections the measured intensity is the impulse train convolved with the squared magnitude of the window transform, h_crystal(q) = |W(q)|². The finite size of the crystal is therefore not an empirical correction applied to the physics; it is a linear shift-invariant (LTI) filter applied to the ideal impulse spectrum. Everything that follows is a property of that filter.

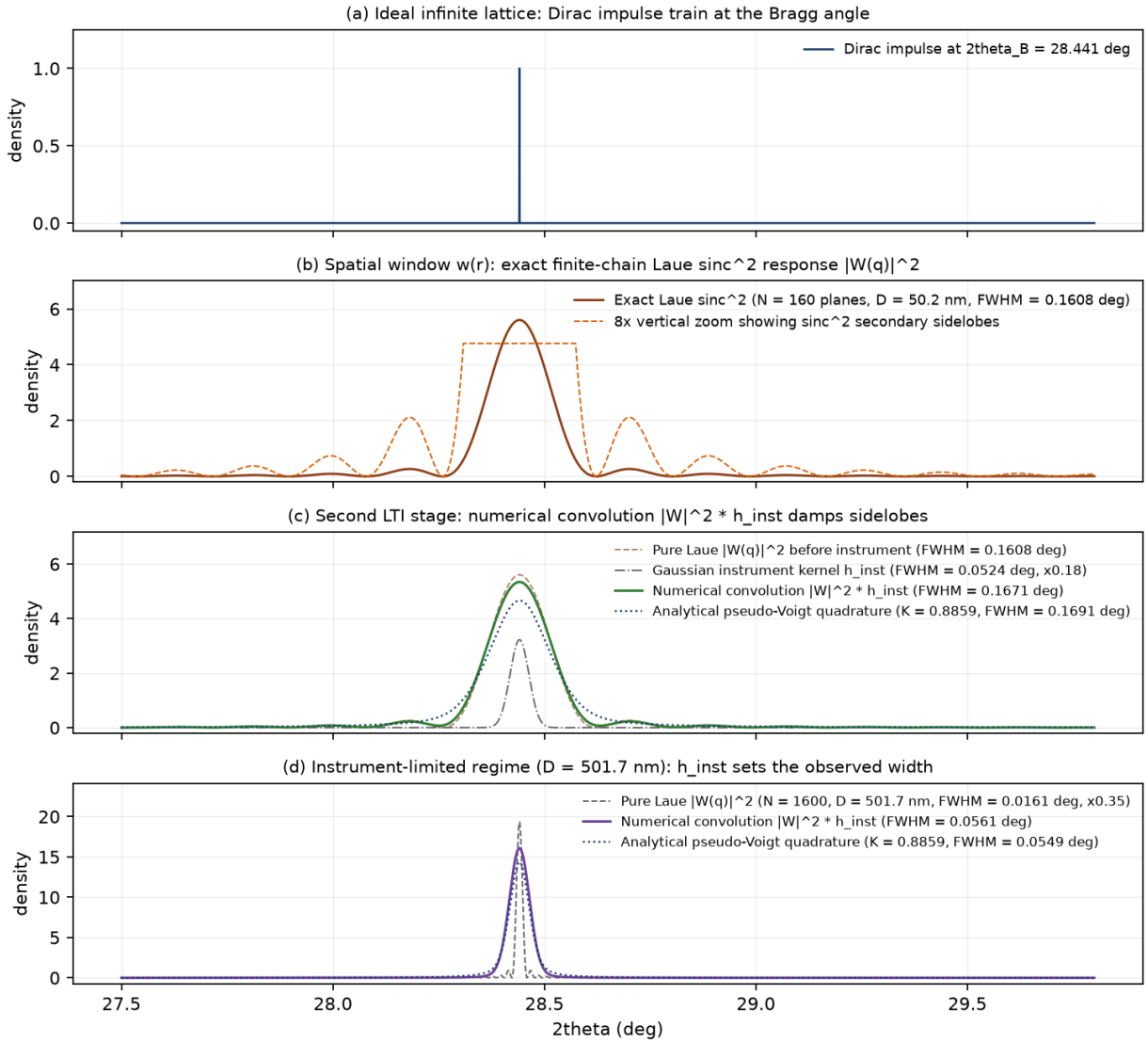


*Figure 7: the signals-and-systems cascade and independent numerical convolution check for Si (111) at N = 160 planes (D = 50.2 nm) and N = 1600 planes (D = 501.7 nm). (a) Ideal Bragg impulse at 2θ_B = 28.441°. (b) Exact finite-chain Laue interference profile h_crystal = |W(q)|² = |sin(Nu)/(N sin u)|² for N = 160 planes (D = 50.2 nm, FWHM = 0.1608°), with secondary sinc² sidelobes shown at 8× vertical zoom. (c) Second LTI stage at N = 160 planes: pure Laue |W(q)|² (FWHM = 0.1608°), compact Gaussian instrument kernel h_inst (FWHM = 0.0524°, scaled ×0.18), direct numerical convolution |W|² * h_inst (FWHM = 0.1671°, damping the sidelobes into smooth wings), and the analytical pseudo-Voigt quadrature curve (K = 0.8859, FWHM = 0.1691°). (d) Instrument-limited regime at N = 1600 planes (D = 501.7 nm), where the narrow Laue*

profile (FWHM = 0.0161°, scaled ×0.35) broadens under numerical convolution to 0.0561° (analytical pseudo-Voigt quadrature 0.0549°), dominated by h_inst.

5.2 The filter is the transform of a rectangle

Take a crystal of N planes with spacing d, so D = N d, and write the window as a rectangle of that length. The intensity near a Bragg point is the squared magnitude of the finite geometric sum, the interference function

```
I(q) ∝ |sin(N q d / 2) / sin(q d / 2)|²,    (2)
```

which reduces to a sinc-squared envelope, width proportional to 1/N, as soon as q is small enough that sin(qd/2) is close to qd/2. Its first zeros sit at q = 2π / D, and its full width at half maximum is a fixed multiple of that scale: the numerical value is 0.886 × 2π / D for large N, 0.890 for N = 10 (Table 7). That number is the shape constant K of the Scherrer relation. The exact value for a finite chain is 0.8859, and the conventional 0.9 is it rounded. K is not a constant of nature: Scherrer's own 1918 paper gives 0.94 for a cube [10], the value depends on the shape of the crystallite and on which reflection is used [7], and it differs again if the width is defined as an integral breadth instead of a half width [8]. Using K = 0.9 against the exact 0.886 therefore returns a size 1.6 per cent too large, which is well inside the scatter between the conventions.

The same result is available without writing the sum: the transform of a rectangle of length D is a sinc whose first zeros are at q = 2π / D, and the squared magnitude gives the observed profile [5]. The two routes must agree, and they do to the numerical values in section 5.3.

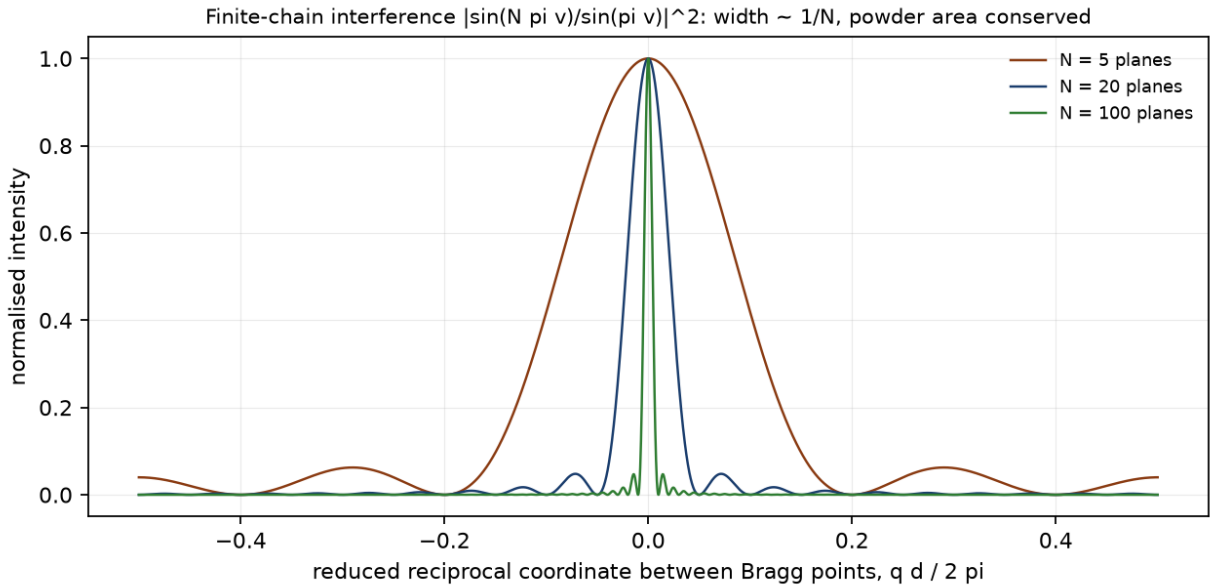


Figure 8: the interference function of equation (2) for N = 5, 20 and 100 planes, plotted against the reduced reciprocal coordinate q d / 2π, that is the distance between Bragg points measured in reciprocal-lattice units. For an isolated crystal the peak height scales as N² while the width scales as 1/N. In a fixed powder specimen volume, the number of crystallites is proportional to 1/N, so the observed peak height scales as N and the integrated powder diffraction area is conserved, exactly as predicted by Parseval's theorem.

5.3 The width constant, measured

Equation (2) can be evaluated directly. Table 7 reports the exact dimensionless FWHM of the finite-chain interference function in reciprocal space, K_N = FWHM_q × D / (2π), together with the corresponding Si (111) pure-size FWHM in degrees and the relative offset from the conventional K = 0.90:

N planes	D at Si 111 (nm)	K_N = FWHM_q D / (2π)	Si 111 FWHM (°)	offset vs K = 0.90
10	3.14	0.8897	2.5798	−1.14 %
20	6.27	0.8869	1.2858	−1.46 %
50	15.68	0.8860	0.5138	−1.56 %

N planes	D at Si 111 (nm)	K_N = FWHM_q D / (2π)	Si 111 FWHM (°)	offset vs K = 0.90
100	31.36	0.8859	0.2569	−1.57 %
1000	313.57	0.8859	0.0257	−1.57 %

Table 7: exact dimensionless shape constant K_N of a finite-chain Laue profile computed from equation (2), the corresponding Si (111) width, and its percentage offset from the conventional K = 0.90 [1].

5.4 Two linear stages, and what follows from that

The intensity chain is two linear, shift-invariant (LTI) stages in series. The input is the ideal Bragg impulse train I_ideal; the finite crystal has intensity impulse response h_crystal = |W|²; the diffractometer optics and detector have impulse response h_instrument. Convolution commutes and associates, so the observed line profile is the ideal impulse blurred by both filters in series [4]:

$$I_{observed} = I_{ideal} * h_{crystal} * h_{instrument}, \quad h_{crystal} = |W|^2. \quad (3)$$

Bandwidth. A wide spatial window has a narrow spatial-frequency response. A crystallite of N planes is a rectangular window of length D = N d, and the width of its squared Fourier transform |W(q)|² scales as 1/D. Halving the crystallite size doubles the spectral width in q, producing the straight line of slope +1 against 1/D (slope −1 against D) in Figure 5. This reciprocal scaling is the spatial analogue of the time-bandwidth uncertainty product in signals and systems [4], [5]: a short spatial window cannot resolve fine spatial-frequency structure, so a small crystal cannot produce a sharp Bragg line.

Area conservation (Parseval's theorem and LTI zero-frequency gain). Two distinct theorems govern the area in equation (3). First, in the first stage, Parseval's theorem applied to the spatial window w(r) of a single crystal of N planes gives $\int |W(q)|^2 \, dq \propto N$; in a fixed irradiated powder volume containing N_cryst ∝ 1/N crystallites, the product is independent of N, so the integrated intensity entering the instrument is invariant with respect to domain size D (which the simulator represents by normalising its crystal profile to unit area). Second, in the second stage, convolving with a unit-area instrument kernel ($\int h_{instrument} \, d(2\theta) = 1$, i.e., unit DC gain) preserves the total integral: $\int (f * g) = (\int f)(\int g)$ [4]. Because total area is conserved across both stages, peak height scales inversely with peak width.

The cascade is limited by its widest stage. The instrument response is set by the slits, the source size and the detector, and for any crystallite above roughly 100 nm it is wider than the crystal response. That is why the curves in Figure 5 flatten at the right, and why the recovery in Table 4 stops being meaningful there: the measurement can no longer see the narrower of the two filters. Adding two Gaussian widths in quadrature is the statement that variances add when independent Gaussian responses are convolved; for a pseudo-Voigt profile or a sinc² Laue profile, quadrature width addition is an analytical approximation (tested directly against exact numerical convolution in Section 5.7).

5.5 The 1/cos(θ) is a change of variable

The broadening happens in q, but the scan is recorded in 2θ. The two are related by q = 4π sin(θ)/λ, so at fixed wavelength λ the Jacobian of the coordinate transformation is

$$dq / d(2\theta) = (2\pi / \lambda) \cos(\theta), \quad (4)$$

and a fixed width in q appears in the scan as

$$\Delta(2\theta) = \Delta(q) / |dq/d(2\theta)| = K \lambda / (D \cos \theta). \quad (5)$$

The 1/cos(theta) is a Jacobian: the change of variable between the frequency axis of the spectrum and the goniometer angle on which it is displayed. At high angle a degree of 2theta spans fewer reciprocal-space units, so the same spectral width looks wider on the chart. (By contrast, differentiating Bragg's law with respect to lambda at fixed d gives the K-alpha1/K-alpha2 wavelength-doublet splitting Delta(2theta) = 2 tan(theta) (Delta lambda / lambda), which shares the 1/cos(theta) projection factor multiplied by sin(theta) and grows from 0.072 deg at 28 deg to 0.275 deg at 88 deg.) This Jacobian is why a size measured from one low-angle line and one high-angle line agrees once equation (5) is applied, as demonstrated in Table 6.

5.6 Sampling, and why the scan step matters

The detector samples the profile at the step size of the scan. A peak measured with too few points per width is undersampled in exactly the sense of the sampling theorem: its height and its width become functions of where the samples happen to fall, and a fit to three points does not recover the continuous line. In the study the narrowest line is the 500 nm case, whose width is about 0.055 deg; a 0.02 deg step gives under three points per FWHM, and the measured width then depends on the step. The 0.002 deg step used for Table 4 is chosen to sample the narrowest line by about 27 points, which is why the numbers in that table can be compared with each other.

This is the same trade-off as bandwidth against acquisition time in any measurement: sampling finer costs counts per point or time, and sampling coarser throws away the information the width carries. The size of the effect is measurable: at 500 nm the same scan taken at a 0.02 deg step reports a width 4.2 per cent larger and a size 28.5 per cent smaller than the 0.002 deg scan, while at 0.002 deg the error is 0.3 per cent. A real experiment that wants a size below 10 nm must choose its step from the expected width, not from habit.

5.7 Pipeline verification and independent Laue convolution check

It is important to distinguish two levels of verification in this study. First, comparing Table 5 against equation (5) verifies the numerical consistency of the simulation-and-measurement pipeline: because the scan generator sets its pseudo-Voigt width by combining equation (5) (with K = 0.90) and the Caglioti width in quadrature, measuring the FWHM on the simulated scan tests how accurately the peak-finding routine recovers the injected width parameter. At 20 nm the pure-size prediction for 111 is 0.410 deg (quadrature total 0.413 deg) against 0.413 deg measured; at 400, 0.482 deg (quadrature total 0.486 deg) against 0.485 deg; at 5 nm, 1.64 deg against 1.64 deg.

Second, to provide a strictly non-circular physical test that never invokes the Scherrer equation or quadrature width addition, `xrdsim/profile.py` evaluates the exact N-plane Laue interference sum h_crystal = |W(q)|^2 = |sin(Nu)/(N sin u)|^2 directly on the 2theta grid (`laue_interference_array`) and numerically convolves it with a Gaussian instrumental kernel h_inst (FWHM = 0.0524 deg at Si 111; `convolve_laue_with_instrument`, plotted in Figure 7(b)-(d) for N = 160 and N = 1600 planes). A Gaussian kernel (eta = 0) is used for h_inst in this discrete convolution benchmark so that the kernel has compact numerical support (decaying to 10^-16 at +/-6 sigma without finite-window Lorentzian tail truncation) and directly tests the Gaussian variance-addition hypothesis of quadrature width combination. Across the entire size range, the numerically convolved Laue FWHM agrees with the analytical finite-chain quadrature formula (K = 0.8859) to within 0.2-2.2 per cent: at N = 16 planes (D = 5.02 nm), 1.6062 deg against 1.6088 deg (0.16 %, or 1.6343 deg at K = 0.90); at N = 64 planes (D = 20.07 nm), 0.4044 deg against 0.4054 deg (0.25 %); at N = 160 planes (D = 50.17 nm, Figure 7(b)-(c)), 0.1671 deg against 0.1691 deg (1.2 %); and at N = 1600 planes (D = 501.7 nm, Figure 7(d)), the 0.0161 deg Laue profile broadens under numerical convolution to 0.0561 deg against 0.0549 deg in quadrature (2.2 %), confirming the instrument-limited floor from first principles.

Two other descriptions of the same physics are common and are worth naming, because a reader will meet them. The Warren-Averbach analysis works with the Fourier coefficients of the line profile directly, which is the most literal form of section 5.1 and separates size from strain by their different order dependence [2]. It exists because a single breadth cannot do that on its own: the breadth gives a mean dimension, and the shape of the size distribution appears only in the higher-order coefficients [9]. The Williamson-Hall plot uses several reflections to separate the 1/cos and tan terms; the analysis code in this repository implements it as a check and returns the input size and zero strain on a size-only simulation. Neither is needed for item 3, and neither is used for the numbers in this report, but both are the same convolution argument written in a different basis.

6. Verification, uncertainty and limits

Every number in this report is produced by the simulator that draws the figures, and all figures, tables, and the PDF rebuild reproducibly from the scripts in Appendix A. Silicon peak positions match the NIST SRM 640f certified lattice parameter (a = 5.431144 A) [14], and every phase in the library matches its published strongest reflection [3], [12]. In the companion GitHub repository, an automated test suite checks the lattice geometry, the structure factor and its systematic absences, the profile functions (including the exact Laue numerical convolution), the detector, and the scan, and includes an independent brute-force count of the

symmetry orbits and a metric-tensor check of the d-spacings.

The uncertainty in the size numbers has four parts. First, the deterministic size sweeps in Table 4 and Table 5 are run without random counting noise so that the analytical profile widths can be read without stochastic scatter. Setting the integrated scale of the strongest reflection to 100 000 counts yields peak heights between 96 counts (at 5 nm) and 2869 counts (at 500 nm) in Table 4. When Poisson counting statistics are enabled, the relative noise on the integrated peak area is 1/sqrt(100 000) = 0.32 per cent, whereas the single-bin noise at the peak maximum of the 500 nm (111) line is 1/sqrt(2869) = 1.9 per cent (and 10.2 per cent per bin on the broad 5 nm profile, where fitting over ~800 points across the FWHM restores sub-per-cent precision). Second, the profile shape matters less than one might expect: sweeping eta from 0 to 1 changes the peak height by about a third but the measured FWHM by less than 0.1 per cent at 20 nm, because the width is read at half maximum and the mixture is normalised to unit area. Third, Lorentzian wing overlap from neighbouring reflections introduces up to a -1.9 % width bias on the weak Si 400 peak at 5 nm (while all reflections at D >= 10 nm agree within 0.5 %), and replacing analytical quadrature with exact numerical Laue convolution shifts the total width by 0.2-2.2 % (Section 5.7). Finally, the instrument constants U, V and W are plausible defaults rather than a calibration of a real diffractometer, so the absolute widths belong to this model; the scaling laws, which are what items 2 and 3 are about, do not depend on them.

What could change the conclusions? A different shape constant would move every recovered size by the same factor and would not change the 1/D law. A different instrument width would move the crossover between the sample-limited and instrument-limited regimes, but not the slope. A specimen with strain would broaden the lines with a tan(theta) term, which the size-only analysis would misread as a smaller crystallite at high angle; that is precisely what the Williamson-Hall check in the repository is for, and why this report uses a strain-free simulation.

Deliberately not modelled: Rietveld refinement, anomalous dispersion, preferred orientation, K-beta residual, detector energy discrimination, thin-film absorption, axial-divergence asymmetry and absolute intensities. Every site shares one Debye-Waller coefficient. The Cu K-alpha wavelengths are the conventional values; no primary table was read for them. None of these affects the size argument, which rests on the width and on the convolution.

6.1 Uncertainty budget

The table collects the terms discussed above, each measured on this model rather than estimated. The two that matter for a size measurement are the scan step on a narrow line and the instrument width itself; the others are below the differences the report is asked to demonstrate.

Source	Effect measured	Where it matters
Counting noise (when enabled)	0.32 % on integrated area; 1.9 % on peak bin at 500 nm	noise-free in Table 4/5
Profile shape, eta 0 to 1	width below 0.1 %, height about 32 %	height comparisons, not widths
Scan step, 0.02 deg at 500 nm	width +4.2 %, recovered size -28.5 %	narrow lines, large crystallites
Neighbour wing overlap	-1.9 % on extracted FWHM at 5 nm, 400 (0.0-0.5 % for D >= 10 nm)	weak broad peaks near stronger lines (Table 5)
Exact Laue vs quadrature width	0.2 % (at 5-20 nm) to 2.2 % (at 500 nm)	intermediate/instrument crossover (Fig. 7)
Instrument U, V, W	sets the crossover near 100 nm	any size above about 100 nm
Wavelength convention	0.0001 deg at 30 deg	negligible
Cell constants	below 0.02 deg in 2theta	positions, not widths

Table 8: the uncertainty budget. The step and eta entries were measured by sweeping the parameter and re-running the analysis; the wing-overlap entry is the largest residual in Table 5, and the Laue convolution entry is from Section 5.7.

7. Conclusion

Item 1: the simulator described in section 2 reproduces the diffraction patterns of canonical crystal types (generic SC, BCC, FCC, amorphous), lattice constant sweeps (--a), nine reference phases in the sample library, and one structure across seven crystallite sizes from 5 nm to 500 nm (Figures 2, 3, and 4), and its complete Python source code is included in Appendix A. Item 2: in the sample-dominated regime (D <~ 100 nm), changing the domain size changes the width of every line as 1/D and the peak height in proportion in a fixed powder volume (before approaching the instrumental resolution floor sqrt(beta_size^2 + beta_inst^2) at larger D), leaves the integrated area and the Bragg positions unchanged, and broadens lines more strongly at high angle as 1/cos(theta). Item 3: those four facts are the properties of a two-stage linear filter cascade. A finite crystallite multiplies the infinite lattice by a spatial window w(r), convolution in reciprocal space turns that window into the Laue interference intensity filter h_crystal = |W(q)|^2 of width 0.8859 x 2 pi / D, the instrument is a second convolution in series with h_inst, and the 1/cos(theta) factor is the Jacobian between reciprocal space and the goniometer angle. Both the pseudo-Voigt scan measurements (Tables 4 and 5) and the independent numerical convolution of the exact N-plane Laue sum with h_inst (Figure 7 and Section 5.7) quantitatively confirm this description.

References

[1] A. L. Patterson, *The Scherrer formula for X-ray particle size determination*, Physical Review **56**, 978-982 (1939). doi:10.1103/PhysRev.56.978

[2] B. E. Warren, *X-ray Diffraction*, Addison-Wesley (1969); Dover reprint (1990), ISBN 0-486-66317-5.

[3] B. D. Cullity and S. R. Stock, *Elements of X-ray Diffraction*, 3rd edition, Prentice Hall (2001), ISBN 0-201-61091-4.

[4] A. V. Oppenheim and A. S. Willsky, *Signals and Systems*, 2nd edition, Prentice Hall (1997), ISBN 0-13-814757-4.

[5] R. N. Bracewell, *The Fourier Transform and Its Applications*, 3rd edition, McGraw-Hill (2000), ISBN 0-07-303938-1.

[6] D. Waasmaier and A. Kirfel, *New analytical scattering-factor functions for free atoms and ions*, Acta Crystallographica **A51**, 416-431 (1995). doi:10.1107/S0108767394013292

[7] J. I. Langford and A. J. C. Wilson, *Scherrer after sixty years: a survey and some new results in the determination of crystallite size*, Journal of Applied Crystallography **11**, 102-113 (1978). doi:10.1107/S0021889878012844

[8] A. R. Stokes and A. J. C. Wilson, *A method of calculating the integral breadths of Debye-Scherrer lines*, Proceedings of the Cambridge Philosophical Society **38**, 313-322 (1942). doi:10.1017/S0305004100021988

[9] F. Bertaut, *Raies de Debye-Scherrer et repartition des dimensions des domaines de Bragg dans les poudres polycristallines*, Acta Crystallographica **3**, 14-18 (1950). doi:10.1107/S0365110X50000045

[10] P. Scherrer, *Bestimmung der Größe und der inneren Struktur von Kolloidteilchen mittels Röntgenstrahlen*, Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse **1918**, 98-100 (1918).

[11] B. E. Warren, *The Diffraction of X-Rays in Glass*, Physical Review **45**(10), 657-661 (1934). doi:10.1103/PhysRev.45.657

[12] S. Gražulis, D. Chateigner, R. T. Downs, A. F. T. Yokochi, M. Quirós, L. Lutterotti, E. Manakova, J. Butkus, P. Moeck and A. Le Bail, *Crystallography Open Database – an open-access collection of crystal structures*, Journal of Applied Crystallography **42**(4), 726-729 (2009). doi:10.1107/S0021889809016690

[13] S. Gates-Rector and T. Blanton, *The Powder Diffraction File: a quality materials characterization database*, Powder Diffraction **34**(4), 352-360 (2019). doi:10.1017/S0885715619000812

[14] National Institute of Standards and Technology, *Certificate of Analysis, Standard Reference Material 640f: Line Position and Line Shape Standard for Powder Diffraction (Silicon Powder)*, Gaithersburg, MD: NIST (2019).

Appendix A: the code

This appendix contains all 14 Python source modules of the `xrdsim/` simulation package (in alphabetical order, including `xrdsim/atoms.py` which loads and evaluates the five-Gaussian Waasmaier-Kirfel atomic form-factor parameterisation [6]) followed by the three analysis and figure-generation scripts `report/size_study.py`, `report/tables.py`, and `report/make_figures.py`. Every source line is formatted to at most 94 characters so that all 17 modules can be copied directly without line-wrapping breaks; the static Waasmaier-Kirfel coefficient table (`data/scattering_factors.json`) and the automated unit test suite (`tests/`) are available in the project directory at

https://github.com/OBODA0/quantum-physics/tree/main/assignments/02-xrd-diffractometer.

xrdsim/__init__.py

```
"""XRD diffractometer simulator (assignment 2, quantum physics section)."""
__version__ = "0.1.0"
```

xrdsim/__main__.py

```
from .cli import main

raise SystemExit(main())
```

xrdsim/analysis.py

```
"""Reading a simulated or measured pattern: peak finding, widths and indexing.

The functions here work on arrays (2theta, counts) and make no assumption about where
they came from, so the same code is used on simulator output and on a measured scan.
"""

from __future__ import annotations

import math
from dataclasses import dataclass

import numpy as np

from .profile import scherrer_size


@dataclass(frozen=True)
class Peak:
    two_theta: float
    height: float
    fwhm: float
    area: float


def find_peaks(two_theta: np.ndarray, counts: np.ndarray,
               min_height_fraction: float = 0.02) -> list[Peak]:
    """Local maxima above a fraction of the tallest point, with width and area."""
    x = np.asarray(two_theta, dtype=float)
    y = np.asarray(counts, dtype=float)
    if x.size != y.size or x.size < 3:
        raise ValueError("need matching arrays of at least three points")
    threshold = min_height_fraction * y.max()
    peaks: list[Peak] = []
    for i in range(1, len(y) - 1):
        if not (y[i] >= y[i - 1] and y[i] > y[i + 1] and y[i] >= threshold):
            continue
        centre, height = _refine(x, y, i)
        floor = _local_floor(x, y, i)
        half = (height + floor) / 2.0
        fwhm = _width_at(x, y, i, half)
        area = _area(x, y, i, 8.0 * fwhm if fwhm > 0 else x[1] - x[0], floor)
        peaks.append(Peak(centre, height, fwhm, area))
    return peaks


def _local_floor(x: np.ndarray, y: np.ndarray, i: int, reach_deg: float = 15.0) -> float:
    """Baseline near a peak: the 2nd percentile within `reach_deg`, so neither the peak's
    own Lorentzian flanks nor a neighbouring reflection biases the baseline."""
    step = x[1] - x[0] if len(x) > 1 else 1.0
    reach = max(1, int(reach_deg / step))
    left = max(0, i - reach)
    right = min(len(y), i + reach)
    return float(np.percentile(y[left:right], 2))


def _width_at(x: np.ndarray, y: np.ndarray, i: int, half: float) -> float:
    left = i
    while left > 0 and y[left] > half:
        left -= 1
    right = i
    while right < len(y) - 1 and y[right] > half:
        right += 1
    if left == i or right == i:
        return 0.0
    x_left = np.interp(half, [y[left], y[left + 1]], [x[left], x[left + 1]])
    x_right = np.interp(half, [y[right], y[right - 1]], [x[right], x[right - 1]])
    return float(x_right - x_left)
```

```
def _area(x: np.ndarray, y: np.ndarray, i: int, half_window: float,
          floor: float = 0.0) -> float:
    window = np.abs(x - x[i]) <= half_window
    trapz = getattr(np, "trapezoid", getattr(np, "trapz", None))
    return float(trapz(np.clip(y[window] - floor, 0.0, None), x[window]))


def pure_width(observed: float, instrumental: float) -> float:
    """Sample-only FWHM by Gaussian quadrature subtraction, guarded at zero."""
    value = observed * observed - instrumental * instrumental
    return math.sqrt(value) if value > 0 else 0.0


def scherrer_from_peak(peak: Peak, instrumental_fwhm: float, wavelength: float,
                       k: float = 0.9) -> float:
    """Crystallite size in angstrom from one peak, corrected for instrumental width."""
    beta = pure_width(peak.fwhm, instrumental_fwhm)
    if beta <= 0:
        raise ValueError("the peak is not broader than the instrument")
    return scherrer_size(beta, peak.two_theta, wavelength, k)


def index_cubic(
    two_thetas: list[float], max_n_squared: int = 30, tolerance: float = 0.02
) -> list[tuple[tuple[int, int, int], float]]:
    """Assign (hkl) to a cubic pattern by the sin^2(theta) ratios.

    The cell returned is the smallest one that fits: when fewer than seven reflections are
    present, a body-centred pattern shares the 1:2:3:4:5:6 ratio sequence of a primitive
    cell with edge a_BCC / sqrt(2) (until the 7th reflection 321 at N^2 = 14 forces
    n2_first = 2); `lattice_hint` notes this ambiguity.
    """
    if not two_thetas:
        raise ValueError("no peaks to index")
    two_thetas = sorted(two_thetas)
    s2 = [math.sin(math.radians(t / 2.0)) ** 2 for t in two_thetas]
    base = s2[0]
    for n2_first in range(1, max_n_squared + 1):
        ratios = [value / base * n2_first for value in s2]
        if all(abs(r - round(r)) <= tolerance * max(round(r), 1) for r in ratios):
            assignments = []
            try:
                for angle, ratio in zip(two_thetas, ratios):
                    n2 = int(round(ratio))
                    assignments.append((_hkl_from_n2(n2), angle))
            except ValueError:
                continue
            return assignments
    raise ValueError("the pattern does not index as cubic")


def lattice_hint(n_squared: list[int]) -> str:
    """Describe the lattice from the observed n^2 sequence.

    All-even n^2 means a body-centred lattice; the FCC sequence starts 3, 4, 8, 11, 12,
    16, 19, 20, 24. A 1, 2, 3, ... sequence is primitive cubic (or BCC with conventional
    cell edge sqrt(2) times this primitive spacing when fewer than 7 lines are observed).
    """
    values = sorted({int(value) for value in n_squared})
    fcc = {3, 4, 8, 11, 12, 16, 19, 20, 24, 27, 32, 35, 36, 40, 43, 44, 48}
    if values and all(value % 2 == 0 for value in values):
        return "body-centred (even N^2 sequence: 2, 4, 6, ...)"
    if set(values) <= fcc:
        return "face-centred (the 3, 4, 8, 11 sequence)"
    if 1 in values:
        return "primitive cell (or BCC with conventional edge sqrt(2) times this spacing)"
    return "primitive lattice"


def _hkl_from_n2(n2: int) -> tuple[int, int, int]:
    for h in range(0, 8):
        for k in range(0, 8):
            l_sq = n2 - h * h - k * k
            if l_sq < 0:
                continue
            l = round(math.isqrt(l_sq))
            if l * l == l_sq:
                return tuple(sorted((h, k, l), reverse=True))
    raise ValueError(f"no (hkl) has n^2 = {n2}")


def _refine(x: np.ndarray, y: np.ndarray, i: int) -> tuple[float, float]:
    """Parabolic refinement of a sampled maximum: position and height between bins."""
    y0, y1, y2 = y[i - 1], y[i], y[i + 1]
```

```
    denom = y0 - 2.0 * y1 + y2
    if denom == 0:
        return float(x[i]), float(y[i])
    delta = 0.5 * (y0 - y2) / denom
    if abs(delta) > 0.5:
        return float(x[i]), float(y[i])
    pos = float(x[i] + delta * (x[i + 1] - x[i]))
    height = float(y1 - 0.25 * (y0 - y2) * delta)
    return pos, height


def williamson_hall(peaks: list[Peak], instrumental_fwhm_for,
                    wavelength: float, k: float = 0.9) -> tuple[float, float]:
    """Crystallite size and microstrain from the Williamson-Hall plot.

    beta cos(theta) = K lambda / D + 4 epsilon sin(theta), fitted by least squares over the
    peaks given. `instrumental_fwhm_for(two_theta)` returns the instrument width at that
    angle. Returns (size in angstrom, strain) and raises when fewer than two peaks are
    usable.
    """
    xs, ys = [], []
    for peak in peaks:
        beta = pure_width(peak.fwhm, instrumental_fwhm_for(peak.two_theta))
        if beta <= 0 or peak.two_theta <= 0:
            continue
        theta = math.radians(peak.two_theta / 2.0)
        xs.append(4.0 * math.sin(theta))
        ys.append(math.radians(beta) * math.cos(theta))
    if len(xs) < 2:
        raise ValueError(
            "Williamson-Hall needs at least two peaks broader than the instrument"
        )
    slope, intercept = np.polyfit(xs, ys, 1)
    if intercept <= 0:
        raise ValueError("the fit has no positive intercept, so no size can be reported")
    return k * wavelength / intercept, float(slope)


def identify_phase(two_thetas: list[float], phase, instrument,
                   tolerance: float = 0.15,
                   min_intensity_fraction: float = 0.15) -> float:
    """Fraction of a phase's strong reflections that the observed peaks contain.

    A score of 1.0 means every predicted line above `min_intensity_fraction` of the
    phase's strongest has an observed peak within `tolerance` degrees.
    """
    from .diffractometer import reflections
    rows = reflections(phase, instrument)
    if not rows:
        return 0.0
    strongest = max(row.intensity for row in rows)
    predicted = [row.two_theta for row in rows
                 if row.intensity > min_intensity_fraction * strongest]
    if not predicted or not two_thetas:
        return 0.0
    matched = sum(1 for angle in predicted
                  if min(abs(angle - seen) for seen in two_thetas) <= tolerance)
    return matched / len(predicted)


def identify(two_thetas: list[float], phases: dict, instrument,
             tolerance: float = 0.15) -> list[tuple[str, float]]:
    """Rank database phases by how well they explain the observed peak positions."""
    scored = [(key, identify_phase(two_thetas, phase, instrument, tolerance))
              for key, phase in phases.items()]
    return sorted(scored, key=lambda row: (-row[1], row[0]))
```

xrdsim/atoms.py

```
"""Atomic scattering factors and the Debye-Waller term.

`f0` is interpolated from data/scattering_factors.json, sampled once from the
Waasmaier-Kirfel analytic form factors (Acta Cryst. A51, 416-431, 1995).
`s = sin(theta) / lambda` in 1/angstrom.
"""

from __future__ import annotations

import json
import math
import pathlib

DATA_PATH = (
```

```
    pathlib.Path(__file__).resolve().parent.parent / "data" / "scattering_factors.json"
)

_table: dict | None = None


def _load() -> dict:
    global _table
    if _table is None:
        payload = json.loads(DATA_PATH.read_text(encoding="utf-8"))
        _table = {
            "s_max": payload["grid_s_inv_A"][-1],
            "step": payload["grid_s_inv_A"][1] - payload["grid_s_inv_A"][0],
            "elements": payload["f0"],
        }
    return _table


def f0(element: str, s: float) -> float:
    """Elastic form factor f0 in electrons. `s = sin(theta)/lambda` in 1/angstrom."""
    table = _load()
    values = table["elements"].get(element)
    if values is None:
        raise KeyError(
            f"no scattering factor for {element!r}; rebuild data/scattering_factors.json"
        )
    if s < 0:
        raise ValueError("s must be non-negative")
    position = s / table["step"]
    last = len(values) - 1
    if position >= last:
        return float(values[last])
    i = int(position)
    frac = position - i
    return float(values[i] * (1.0 - frac) + values[i + 1] * frac)


def debye_waller(b_iso: float, s: float) -> float:
    """Temperature factor exp(-B s^2), with B = 8 pi^2 <u^2> in square angstrom."""
    if b_iso < 0:
        raise ValueError("B must be non-negative")
    return math.exp(-b_iso * s * s)
```

xrdsim/cli.py

```
"""Command line interface: `python -m xrdsim`."""

from __future__ import annotations

import argparse
import dataclasses
import pathlib

from . import sources
from .analysis import (find_peaks, identify, index_cubic, lattice_hint,
                       scherrer_from_peak, williamson_hall)
from .diffractometer import Instrument, amorphous_scan, reflections, scan
from .profile import caglioti_fwhm, scherrer_size
from .samples import PHASES, get_phase, simple_cubic


def _instrument(args) -> Instrument:
    return Instrument(
        ka2=not args.no_ka2,
        monochromator_two_theta=args.monochromator,
        reference_counts=args.counts,
        background_rate=args.background,
        dwell_s=args.dwell,
        rng_seed=None if args.noise == 0 else args.noise,
        sample_displacement_mm=args.displacement,
    )


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(prog="xrdsim", description=__doc__)
    parser.add_argument("--phase", help="phase key (see --list-phases)")
    parser.add_argument("--a", type=float, default=None,
                        help="override lattice constant a in angstrom")
    parser.add_argument("--range", nargs=2, type=float, metavar=("MIN", "MAX"),
                        default=[20.0, 90.0], help="2theta scan range in degrees")
    parser.add_argument("--step", type=float, default=0.02, help="2theta step in degrees")
    parser.add_argument("--dwell", type=float, default=1.0, help="counting time per step, s")
    parser.add_argument("--counts", type=float, default=20_000.0,
```

```
                      help="integrated counts at the strongest reflection")
    parser.add_argument("--background", type=float, default=0.5, help="background, counts/s")
    parser.add_argument("--displacement", type=float, default=0.0, metavar="MM",
                      help="sample surface displaced from the goniometer axis")
    parser.add_argument("--size", type=float, default=None,
                      help="crystallite size in angstrom (adds size broadening)")
    parser.add_argument("--size-nm", type=float, default=None,
                      help="crystallite size in nm (adds size broadening)")
    parser.add_argument("--no-ka2", action="store_true", help="drop the K-alpha2 line")
    parser.add_argument("--monochromator", type=float, default=None,
                      help="monochromator 2theta in degrees")
    parser.add_argument("--noise", type=int, default=0,
                      help="Poisson seed; 0 disables counting noise")
    parser.add_argument("--out", type=pathlib.Path, help="write the scan to this image")
    parser.add_argument("--table", action="store_true", help="print the reflection table")
    parser.add_argument("--index", action="store_true",
                      help="index the observed peaks as a cubic pattern")
    parser.add_argument("--scherrer", action="store_true",
                      help="report Scherrer sizes for the observed peaks")
    parser.add_argument("--identify", action="store_true",
                      help="rank the phase database against the simulated pattern")
    parser.add_argument("--list-phases", action="store_true", help="list the phase database")
    args = parser.parse_args(argv)

    if args.list_phases:
        for key, phase in sorted(PHASES.items()):
            print(f"{key:14s} {phase.formula:8s} {phase.space_group:18s} {phase.source}")
        sc = simple_cubic()
        print(f"{'sc':14s} {sc.formula:8s} {sc.space_group:18s} {sc.source}")
        print(f"{'amorphous':14s} {'SiO2':8s} {'non-crystalline':18s} "
              "phenomenological diffuse silica halo at 21.8 deg")
        return 0
    if not args.phase:
        parser.error("--phase is required unless --list-phases is given")

    try:
        return _run(args)
    except (KeyError, ValueError) as error:
        parser.error(str(error).strip('"'))


def _run(args) -> int:
        instrument = _instrument(args)
        if args.phase == "amorphous":
            result = amorphous_scan(args.range[0], args.range[1], args.step, instrument)
            d_eff = scherrer_size(8.8, 21.8, instrument.wavelength) / 10.0
            print("phase      : Amorphous silica (SiO2), non-crystalline")
            print(f"wavelength : {instrument.wavelength:.6f} A")
            print(f"range      : {args.range[0]} - {args.range[1]} deg, step {args.step}")
            print(f"halo       : center 21.800 deg  FWHM 8.8000 deg  D_eff {d_eff:.2f} nm")
            if args.out:
                from .plot import save_scan
                args.out.parent.mkdir(parents=True, exist_ok=True)
                save_scan(result, args.out, title="Amorphous SiO2, diffuse halo")
                print(f"wrote {args.out}")
            return 0

        phase = get_phase(args.phase)
        if args.a is not None:
            if args.a <= 0:
                raise ValueError("lattice constant --a must be positive")
            phase = dataclasses.replace(phase, a=args.a)
        size_a = args.size_nm * 10.0 if args.size_nm is not None else args.size
        result = scan(phase, args.range[0], args.range[1], args.step, instrument,
                      size_a=size_a)

        if args.table:
            strongest = max(row.intensity for row in result.reflections)
            print(f"{'hkl':>10s} {'d (A)':>8s} {'2theta':>8s} {'m':>3s} {'I/I0':>7s}")
            for row in result.reflections:
                if row.intensity < 1e-4 * strongest:
                    continue
                label = "".join(str(v) for v in row.hkl)
                rel_i = 100 * row.intensity / strongest
                print(f"{label:>10s} {row.d:8.4f} {row.two_theta:8.3f} "
                      f"{row.multiplicity:3d} {rel_i:7.2f}")
            return 0

        peaks = find_peaks(result.two_theta, result.counts)
        print(f"phase      : {phase.name} ({phase.formula}), {phase.space_group}, "
              f"a = {phase.a:.4f} A")
        print(f"wavelength : {instrument.wavelength:.6f} A"
              + (" + K-alpha2" if instrument.ka2 else ""))
        print(f"range      : {args.range[0]} - {args.range[1]} deg, step {args.step}")
```

```
        print(f"peaks      : {len(peaks)}")
        for peak in peaks[:12]:
            line = (
                f"  {peak.two_theta:8.3f} deg  height {peak.height:9.0f}  "
                f"FWHM {peak.fwhm:.4f}"
            )
            if size_a and not args.scherrer:
                try:
                    inst_w = caglioti_fwhm(peak.two_theta, *instrument.caglioti)
                    size = scherrer_from_peak(peak, inst_w, instrument.wavelength)
                    line += f"  Scherrer {size / 10.0:.1f} nm"
                except ValueError:
                    pass
            print(line)

        if args.index:
            try:
                rows = index_cubic([peak.two_theta for peak in peaks])
            except ValueError as error:
                print(f"indexing failed: {error}")
            else:
                print("indexing   : hkl            2theta")
                for hkl, angle in rows:
                    print(f"             {str(hkl):14s} {angle:7.3f}")
                n_squared = [hkl[0] ** 2 + hkl[1] ** 2 + hkl[2] ** 2 for hkl, _ in rows]
                print(f"lattice    : {lattice_hint(n_squared)}")

        if args.scherrer:
            print("Scherrer   : peak            size (nm)")
            for peak in peaks:
                try:
                    inst_w = caglioti_fwhm(peak.two_theta, *instrument.caglioti)
                    size = scherrer_from_peak(peak, inst_w, instrument.wavelength)
                except ValueError:
                    continue
                print(f"             {peak.two_theta:7.3f}         {size / 10.0:6.1f}")
            try:
                wh_size, wh_strain = williamson_hall(
                    peaks, lambda angle: caglioti_fwhm(angle, *instrument.caglioti),
                    instrument.wavelength)
            except ValueError as error:
                print(f"Williamson-Hall: {error}")
            else:
                note = "" if args.no_ka2 else " (use --no-ka2: shoulders bias the fit)"
                print(
                    f"Williamson-Hall: D = {wh_size / 10.0:.1f} nm, "
                    f"strain = {wh_strain:.5f}{note}"
                )

        if args.identify:
            ranking = identify([peak.two_theta for peak in peaks], PHASES, instrument)
            print("identification: phase          score")
            for key, score in ranking[:5]:
                print(f"                {key:14s} {score:5.2f}")

        if args.out:
            from .plot import save_scan
            args.out.parent.mkdir(parents=True, exist_ok=True)
            save_scan(result, args.out,
                      title=f"{phase.name}, {instrument.wavelength:.4f} A")
            print(f"wrote {args.out}")
        return 0
```

xrdsim/detector.py

```
"""Detector: counting statistics and dead time.

A powder scan is a counting experiment. The model returns expected counts per step; the
detector turns them into observed counts with Poisson noise.
"""

from __future__ import annotations

import math


def expected_counts(intensity: float, dwell_s: float) -> float:
    """Mean counts at a step: intensity (counts per second) times dwell time."""
    if dwell_s <= 0:
        raise ValueError("dwell time must be positive")
    return max(intensity, 0.0) * dwell_s
def observe(mean_counts: float, rng) -> float:
    """One Poisson draw for a step. `rng` is a numpy Generator."""
```

```
    return float(rng.poisson(max(mean_counts, 0.0)))


def count_noise(mean_counts: float) -> float:
    """Standard deviation of a Poisson count (sqrt(N)); zero for an empty step."""
    return math.sqrt(max(mean_counts, 0.0))


def apply_dead_time(true_rate: float, tau_s: float) -> float:
    """Observed rate for a non-paralysable detector: n = m / (1 + m tau)."""
    if true_rate < 0 or tau_s < 0:
        raise ValueError("rates and dead time must be non-negative")
    return true_rate / (1.0 + true_rate * tau_s)
```

xrdsim/diffractometer.py

```
"""The diffractometer: reflections, peak profiles and the scan itself.

A scan is built in counts per step. Reflection intensities are relative (I/I0): the
model reproduces diffraction patterns and relative intensity ratios, not absolute
radiometric photon flux.
"""

from __future__ import annotations

import math
from dataclasses import dataclass, field

import numpy as np

from . import sources
from .detector import apply_dead_time, expected_counts
from .lattice import d_spacing
from .profile import (add_in_quadrature, caglioti_fwhm, pseudo_voigt_array,
                      size_fwhm)
from .samples import Phase
from .structure import lorentz_polarisation, multiplicity, structure_factor


@dataclass(frozen=True)
class Instrument:
    wavelength: float = sources.CU_KA1
    ka2: bool = True
    monochromator_two_theta: float | None = None
    caglioti: tuple[float, float, float] = (0.004, -0.002, 0.003)
    eta: float = 0.5
    reference_counts: float = 20_000.0     # counts at the strongest reflection
    background_rate: float = 0.5           # counts per second, flat
    dwell_s: float = 1.0
    dead_time_s: float = 0.0
    rng_seed: int | None = None      # None: no noise; int seeds Poisson draws
    max_two_theta: float = 150.0      # goniometer limit; LP diverges at 180 deg
    sample_displacement_mm: float = 0.0  # specimen surface off goniometer axis
    goniometer_radius_mm: float = 240.0


@dataclass(frozen=True)
class Reflection:
    hkl: tuple[int, int, int]
    d: float
    two_theta: float
    multiplicity: int
    f_squared: float
    intensity: float


@dataclass
class Scan:
    two_theta: np.ndarray
    counts: np.ndarray
    reflections: list[Reflection] = field(default_factory=list)
    phase: Phase | None = None
    instrument: Instrument | None = None



def reflections(phase: Phase, instrument: Instrument, hmax: int = 8) -> list[Reflection]:
    """Every reflection the wavelength reaches, with its relative intensity."""
    rows: list[Reflection] = []
    for h in range(0, hmax + 1):
        for k in range(0, hmax + 1):
            for l in range(0, hmax + 1):
                if h == 0 and k == 0 and l == 0:
```

```
                    continue
                if not _representative(h, k, l, phase.system):
                    continue
                try:
                    d = d_spacing(h, k, l, **phase.cell_kwargs())
                except ValueError:
                    continue
                if 2.0 * d < instrument.wavelength:
                    continue
                two_theta = 2.0 * math.degrees(
                    math.asin(instrument.wavelength / (2.0 * d)))
                if two_theta > instrument.max_two_theta:
                    continue
                f = structure_factor(phase, h, k, l, instrument.wavelength)
                m = multiplicity(h, k, l, phase.system)
                lp = lorentz_polarisation(two_theta, instrument.monochromator_two_theta)
                rows.append(Reflection((h, k, l), d, two_theta, m, abs(f) ** 2,
                                       abs(f) ** 2 * m * lp))
    rows.sort(key=lambda r: r.two_theta)
    return rows


def scan(phase: Phase, two_theta_min: float, two_theta_max: float, step: float,
         instrument: Instrument | None = None, size_a: float | None = None,
         hmax: int = 8) -> Scan:
    """Simulate one 2theta scan, with Poisson noise and optional crystallite size."""
    if instrument is None:
        instrument = Instrument()
    if step <= 0 or two_theta_max <= two_theta_min:
        raise ValueError("the scan range and step must be positive")
    grid = np.arange(two_theta_min, two_theta_max + step / 2.0, step)
    rows = reflections(phase, instrument, hmax)
    if not rows:
        raise ValueError("no reachable reflections in this range")

    strongest = max(row.intensity for row in rows)
    if strongest <= 0:
        raise ValueError("every reflection has zero intensity")

    lines = [(instrument.wavelength, 1.0)]
    if instrument.ka2:
        # the doublet is scaled from the instrument's own K-alpha1 so that another anode
        # keeps its own wavelengths instead of silently getting copper's
        ka2 = instrument.wavelength * sources.CU_KA2 / sources.CU_KA1
        lines = [(instrument.wavelength, 1.0), (ka2, sources.KA2_TO_KA1)]

    bg = expected_counts(instrument.background_rate, instrument.dwell_s)
    mean = np.full_like(grid, bg, dtype=float)
    for row in rows:
        area_total = instrument.reference_counts * row.intensity / strongest
        for wavelength, weight in lines:
            d = d_spacing(*row.hkl, **phase.cell_kwargs())
            centre = 2.0 * math.degrees(math.asin(min(1.0, wavelength / (2.0 * d))))
            centre += displacement_shift(centre, instrument.sample_displacement_mm,
                                         instrument.goniometer_radius_mm)
            width = caglioti_fwhm(centre, *instrument.caglioti)
            if size_a is not None:
                width = add_in_quadrature(width, size_fwhm(size_a, centre, wavelength))
            if not (two_theta_min - 5.0 * width <= centre <= two_theta_max + 5.0 * width):
                continue
            profile = pseudo_voigt_array(grid, centre, width, instrument.eta)
            mean += area_total * weight * profile * step

    if instrument.dead_time_s:
        rate = mean / instrument.dwell_s
        mean = np.array([apply_dead_time(r, instrument.dead_time_s) * instrument.dwell_s
                         for r in rate])

    if instrument.rng_seed is None:
        counts = mean
    else:
        rng = np.random.default_rng(instrument.rng_seed)
        counts = rng.poisson(np.clip(mean, 0.0, None)).astype(float)

    return Scan(grid, counts, rows, phase, instrument)


def _representative(h: int, k: int, l: int, system: str) -> bool:
    """True for one member of each symmetry family.

    Enumerating h, k, l >= 0 visits every permutation of a reflection; the multiplicity
    already counts the whole orbit, so only the canonical member may be used.
    """
    if system == "cubic":
```

```
        return h >= k >= l
    if system == "tetragonal":
        return h >= k
    if system == "hexagonal":
        return h >= k
    return True


def displacement_shift(two_theta_deg: float, displacement_mm: float,
                       radius_mm: float) -> float:
    """2theta shift in degrees for a flat specimen displaced from the goniometer axis.

    Delta(2theta) = -2 s cos(theta) / R, in radians.
    """
    if displacement_mm == 0.0:
        return 0.0
    if radius_mm <= 0:
        raise ValueError('the goniometer radius must be positive')
    theta = math.radians(two_theta_deg / 2.0)
    return math.degrees(-2.0 * displacement_mm * math.cos(theta) / radius_mm)


def amorphous_scan(two_theta_min: float, two_theta_max: float, step: float,
                   instrument: Instrument | None = None,
                   center_deg: float = 21.8, fwhm_deg: float = 8.8) -> Scan:
    """Simulate a phenomenological amorphous silica-like diffuse scattering pattern.

    Models intermediate-range tetrahedral network order (~4.0 A inter-tetrahedral
    correlations) producing a First Sharp Diffraction Peak (FSDP) near 21.8 deg with
    FWHM ~ 8.8 deg (illustrative Scherrer coherence length D_eff ~ 0.9 nm), plus a
    secondary hump near 43 deg, modulated by form-factor decay and polarisation.
    """
    if instrument is None:
        instrument = Instrument()
    if step <= 0 or two_theta_max <= two_theta_min:
        raise ValueError("the scan range and step must be positive")
    grid = np.arange(two_theta_min, two_theta_max + step / 2.0, step)
    s = np.sin(np.radians(grid / 2.0)) / instrument.wavelength
    halo1 = np.exp(-4.0 * math.log(2.0) * ((grid - center_deg) / fwhm_deg) ** 2)
    halo2 = 0.30 * np.exp(-4.0 * math.log(2.0) * ((grid - 43.0) / 16.0) ** 2)
    ff_decay = np.exp(-3.5 * s ** 2)
    pol = 0.5 * (1.0 + np.cos(np.radians(grid)) ** 2)
    envelope = (halo1 + halo2) * ff_decay * pol
    counts = instrument.reference_counts * (envelope / envelope.max())
    return Scan(grid, counts, [], None, instrument)
```

xrdsim/geometry.py

```
"""Bragg-Brentano geometry: sample illumination and absorption.

Angles in degrees, lengths in mm, mass attenuation in cm^2/g. The flat-plate,
infinite-thickness case is the one a laboratory powder scan approximates.
"""

from __future__ import annotations

import math


def bragg_angle(two_theta_deg: float) -> float:
    """theta for a given 2theta."""
    return two_theta_deg / 2.0


def footprint_length(two_theta_deg: float, divergence_slit_mm: float,
                     sample_length_mm: float = 20.0) -> float:
    """Length of sample illuminated by the beam, in mm.

    The beam leaves the divergence slit with a fixed width, so the footprint on the
    flat plate grows as 1/sin(theta) and is clipped by the sample holder.
    """
    theta = math.radians(bragg_angle(two_theta_deg))
    if theta <= 0:
        raise ValueError("2theta must be positive")
    return min(sample_length_mm, divergence_slit_mm / math.sin(theta))


def linear_attenuation(density_g_cm3: float, mass_attenuation_cm2_g: float) -> float:
    """mu in 1/cm."""
    if density_g_cm3 <= 0 or mass_attenuation_cm2_g <= 0:
        raise ValueError("density and mass attenuation must be positive")
    return density_g_cm3 * mass_attenuation_cm2_g
def is_infinitely_thick(
```

```
    mu_per_cm: float, thickness_cm: float, threshold: float = 0.01
) -> bool:
    """True when exp(-2 mu t / sin(theta)) is negligible (worst case theta -> 90 deg)."""
    return math.exp(-2.0 * mu_per_cm * thickness_cm) < threshold


def flat_plate_absorption(mu_per_cm: float) -> float:
    """Absorption factor for a thick flat plate: A = 1 / (2 mu), independent of theta."""
    if mu_per_cm <= 0:
        raise ValueError("mu must be positive")
    return 1.0 / (2.0 * mu_per_cm)


def penetration_depth(
    mu_per_cm: float, two_theta_deg: float, fraction: float = 0.99
) -> float:
    """Depth in cm that carries `fraction` of the diffracted intensity."""
    if not 0.0 < fraction < 1.0:
        raise ValueError("fraction must lie in (0, 1)")
    theta = math.radians(bragg_angle(two_theta_deg))
    return -math.log(1.0 - fraction) * math.sin(theta) / (2.0 * mu_per_cm)
```

xrdsim/lattice.py

```
"""Lattice geometry: d-spacings, Bragg angles and lattice-parameter refinement.

Lengths are in angstrom, angles in degrees. Only the settings a powder scan needs are
supported here: cubic, tetragonal, hexagonal and orthorhombic.
"""

from __future__ import annotations

import math


def d_spacing(h: int, k: int, l: int, *, a: float, b: float | None = None,
              c: float | None = None, hexagonal: bool = False) -> float:
    """Interplanar spacing in angstrom.

    `hexagonal=True` selects the hexagonal setting (a = b, gamma = 120 degrees).
    Otherwise `b` and `c` both None is cubic, `c` given with `b` None is tetragonal, and
    both `b` and `c` given is orthorhombic. A lone `b` is rejected rather than guessed.
    """
    if a <= 0:
        raise ValueError("lattice parameter must be positive")
    if hexagonal:
        if c is None:
            raise ValueError("the hexagonal setting needs c")
        inv_sq = 4.0 / 3.0 * (h * h + h * k + k * k) / a**2 + l * l / c**2
    elif b is None and c is None:
        inv_sq = (h * h + k * k + l * l) / a**2
    elif b is None:
        inv_sq = (h * h + k * k) / a**2 + l * l / c**2
    elif c is None:
        raise ValueError(
            "an orthorhombic cell needs both b and c; "
            "for a tetragonal cell pass c and leave b None"
        )
    else:
        inv_sq = h * h / a**2 + k * k / b**2 + l * l / c**2
    if inv_sq == 0:
        raise ValueError("(0,0,0) has no d-spacing")
    return 1.0 / math.sqrt(inv_sq)


def two_theta(d: float, wavelength: float) -> float:
    """Bragg angle in degrees for a d-spacing, or ValueError if unreachable."""
    if d <= 0:
        raise ValueError("d-spacing must be positive")
    s = wavelength / (2.0 * d)
    if s > 1.0:
        raise ValueError(
            f"d = {d:.4f} A is unreachable at lambda = {wavelength:.4f} A "
            "(needs sin(theta) > 1)"
        )
    return math.degrees(2.0 * math.asin(s))


def d_from_two_theta(two_theta_deg: float, wavelength: float) -> float:
    """d-spacing in angstrom from a measured 2theta, by Bragg's law."""
    theta = math.radians(two_theta_deg / 2.0)
    s = math.sin(theta)
    if s == 0:
```

```
        raise ValueError("2theta = 0 carries no information")
    return wavelength / (2.0 * s)


def refine_cubic_a(
    peaks: list[tuple[float, int, int, int]], wavelength: float
) -> float:
    """Least-squares cubic lattice parameter from indexed peaks.

    `peaks` is a list of (two_theta_deg, h, k, l). The fit is an unweighted least squares
    through the origin, sin^2(theta) = (lambda^2 / 4a^2) * (h^2 + k^2 + l^2): no per-peak
    uncertainties, and no zero-offset term, so a displaced specimen biases the result.
    """
    xs, ys = [], []
    for two_theta_deg, h, k, l in peaks:
        n_sq = h * h + k * k + l * l
        if n_sq == 0:
            raise ValueError("(0,0,0) cannot be indexed")
        xs.append(n_sq)
        ys.append(math.sin(math.radians(two_theta_deg / 2.0)) ** 2)
    denom = sum(x * x for x in xs)
    if denom == 0:
        raise ValueError("no peaks given")
    slope = sum(x * y for x, y in zip(xs, ys)) / denom
    if slope <= 0:
        raise ValueError("degenerate peak list")
    return wavelength / (2.0 * math.sqrt(slope))


def peak_positions(
    hkls, wavelength: float, **cell
) -> list[tuple[tuple[int, int, int], float, float]]:
    """(hkl, d, 2theta) for every reachable reflection, sorted by 2theta."""
    out = []
    for h, k, l in hkls:
        d = d_spacing(h, k, l, **cell)
        try:
            out.append(((h, k, l), d, two_theta(d, wavelength)))
        except ValueError:
            continue
    return sorted(out, key=lambda row: row[2])
```

xrdsim/plot.py

```
"""Matplotlib rendering of a scan.

A scan is drawn as counts against 2theta, with the reflections the model predicts
marked clearly above each peak.
"""

from __future__ import annotations

import matplotlib
import numpy as np

matplotlib.use("Agg")

import matplotlib.pyplot as plt

from .diffractometer import Scan


def plot_scan(scan: Scan, ax=None, title: str | None = None, annotate: bool = True):
    """Draw one scan; returns the axes."""
    if ax is None:
        _, ax = plt.subplots(figsize=(9, 4.2))
    ax.plot(scan.two_theta, scan.counts, lw=0.85, color="#1a3d6d")
    y_max = float(scan.counts.max()) if len(scan.counts) else 1.0
    ax.set_ylim(-0.03 * y_max, 1.22 * y_max)
    if annotate and scan.reflections:
        strongest = max(row.intensity for row in scan.reflections)
        placed: list[float] = []
        for row in sorted(scan.reflections, key=lambda r: -r.intensity):
            if row.intensity < 0.10 * strongest:
                continue
            if not (scan.two_theta[0] <= row.two_theta <= scan.two_theta[-1]):
                continue
            if any(abs(row.two_theta - angle) < 2.2 for angle in placed):
                continue
            placed.append(row.two_theta)
            idx = int(np.argmin(np.abs(scan.two_theta - row.two_theta)))
            y_pos = min(float(scan.counts[idx]), 0.94 * y_max)
            ax.annotate("".join(str(v) for v in row.hkl), (row.two_theta, y_pos),
```

```
                        textcoords="offset points", xytext=(0, 3),
                        fontsize=8.0, fontweight="bold", ha="center", va="bottom",
                        color="#8a3b12")
    ax.set_xlabel("2theta (deg)")
    ax.set_ylabel("counts")
    if title:
        ax.set_title(title)
    ax.grid(alpha=0.25, lw=0.5)
    return ax


def save_scan(scan: Scan, path, title: str | None = None, dpi: int = 160) -> None:
    """Write one scan to an image file."""
    fig, ax = plt.subplots(figsize=(9, 4.2))
    plot_scan(scan, ax=ax, title=title or (scan.phase.name if scan.phase else None))
    fig.tight_layout()
    fig.savefig(path, dpi=dpi)
    plt.close(fig)
```

xrdsim/profile.py

```
"""Peak profiles and the instrument resolution function.

The profile is normalised to unit area, so a peak's height follows from its integrated
intensity and its width. Angles are in degrees, widths are FWHM in degrees.
"""

from __future__ import annotations

import math

TWO_LN2 = 2.0 * math.log(2.0)


def gaussian(x: float, center: float, fwhm: float) -> float:
    """Unit-area Gaussian."""
    sigma = fwhm / (2.0 * math.sqrt(TWO_LN2))
    z = (x - center) / sigma
    return math.exp(-0.5 * z * z) / (sigma * math.sqrt(2.0 * math.pi))


def lorentzian(x: float, center: float, fwhm: float) -> float:
    """Unit-area Lorentzian."""
    half = fwhm / 2.0
    return (half / math.pi) / ((x - center) ** 2 + half * half)


def pseudo_voigt(x: float, center: float, fwhm: float, eta: float) -> float:
    """Unit-area pseudo-Voigt: eta of Lorentzian mixed with (1 - eta) of Gaussian."""
    if not 0.0 <= eta <= 1.0:
        raise ValueError("eta must lie in [0, 1]")
    return eta * lorentzian(x, center, fwhm) + (1.0 - eta) * gaussian(x, center, fwhm)


def caglioti_fwhm(two_theta_deg: float, U: float, V: float, W: float) -> float:
    """Instrumental FWHM in degrees: U tan^2(theta) + V tan(theta) + W."""
    t = math.tan(math.radians(two_theta_deg / 2.0))
    value = U * t * t + V * t + W
    return math.sqrt(max(value, 0.0))


def scherrer_size(fwhm_deg: float, two_theta_deg: float, wavelength: float,
                  k: float = 0.9) -> float:
    """Crystallite size in angstrom from a sample-broadened FWHM (Scherrer).

    `fwhm_deg` must already be corrected for instrumental broadening.
    """
    if fwhm_deg <= 0:
        raise ValueError("FWHM must be positive")
    beta = math.radians(fwhm_deg)
    theta = math.radians(two_theta_deg / 2.0)
    return k * wavelength / (beta * math.cos(theta))


def size_fwhm(size_a: float, two_theta_deg: float, wavelength: float,
              k: float = 0.9) -> float:
    """FWHM in degrees produced by a crystallite of `size_a` angstrom (inverse Scherrer)."""
    if size_a <= 0:
        raise ValueError("size must be positive")
    theta = math.radians(two_theta_deg / 2.0)
    return math.degrees(k * wavelength / (size_a * math.cos(theta)))
def add_in_quadrature(*fwhms: float) -> float:
    """Combine Gaussian widths."""
```

```
    return math.sqrt(sum(f * f for f in fwhms))


def pseudo_voigt_array(x, center: float, fwhm: float, eta: float):
    """Unit-area pseudo-Voigt over an array; same numbers as pseudo_voigt, vectorised."""
    import numpy as np
    if not 0.0 <= eta <= 1.0:
        raise ValueError("eta must lie in [0, 1]")
    values = np.asarray(x, dtype=float)
    sigma = fwhm / (2.0 * math.sqrt(TWO_LN2))
    half = fwhm / 2.0
    z = (values - center) / sigma
    gaussian_part = np.exp(-0.5 * z * z) / (sigma * math.sqrt(2.0 * math.pi))
    lorentzian_part = (half / math.pi) / ((values - center) ** 2 + half * half)
    return eta * lorentzian_part + (1.0 - eta) * gaussian_part


def laue_interference_array(two_theta_deg, two_theta_bragg_deg: float, d_a: float,
                            n_planes: int, wavelength: float):
    """Unit-area exact N-plane Laue profile |sin(N u) / (N sin u)|^2 in 2theta.

    Contains no empirical Scherrer constant K: the width and sinc^2 sidelobes emerge
    solely from summing N coherent atomic planes of spacing `d_a`.
    """
    import numpy as np
    if n_planes < 1:
        raise ValueError("n_planes must be at least 1")
    angles = np.asarray(two_theta_deg, dtype=float)
    theta_b = math.radians(two_theta_bragg_deg / 2.0)
    sin_diff = np.sin(np.radians(angles / 2.0)) - math.sin(theta_b)
    u = 2.0 * math.pi * d_a * sin_diff / wavelength
    denom = n_planes * np.sin(u)
    safe_denom = np.where(np.abs(denom) < 1e-12, 1.0, denom)
    profile = np.where(
        np.abs(denom) < 1e-12, 1.0, (np.sin(n_planes * u) / safe_denom) ** 2
    )
    trapz = getattr(np, "trapezoid", getattr(np, "trapz", None))
    area = float(trapz(profile, angles))
    return profile / area if area > 0 else profile


def convolve_laue_with_instrument(
    two_theta_deg, two_theta_bragg_deg: float, d_a: float,
    n_planes: int, wavelength: float, inst_fwhm_deg: float,
):
    """Direct numerical convolution of the N-plane Laue profile with h_inst."""
    import numpy as np
    angles = np.asarray(two_theta_deg, dtype=float)
    step = float(angles[1] - angles[0])
    laue = laue_interference_array(
        angles, two_theta_bragg_deg, d_a, n_planes, wavelength
    )
    sigma = inst_fwhm_deg / (2.0 * math.sqrt(TWO_LN2))
    half_span = max(0.5, 6.0 * sigma)
    kx = np.arange(-half_span, half_span + step / 2.0, step)
    kernel = np.exp(-0.5 * (kx / sigma) ** 2) / (sigma * math.sqrt(2.0 * math.pi))
    return np.convolve(laue, kernel, mode="same") * step
```

xrdsim/samples.py

```
"""The phase database: cell, symmetry-expanded basis and the source of each number.

Coordinates are fractions of the cell edge. Every site list below is the full basis of
the conventional cell, already expanded by the space-group operations, so the structure
factor needs no symmetry engine. Lattice constants carry the record they came from.
"""

from __future__ import annotations

from dataclasses import dataclass, field


@dataclass(frozen=True)
class Site:
    element: str
    x: float
    y: float
    z: float
    occupancy: float = 1.0
    b_iso: float = 0.5          # square angstrom; a stated assumption for every phase


@dataclass(frozen=True)
```

```
class Phase:
    key: str
    name: str
    formula: str
    space_group: str
    a: float
    c: float | None = None
    b: float | None = None
    hexagonal: bool = False
    sites: tuple[Site, ...] = field(default_factory=tuple)
    source: str = ""

    @property
    def system(self) -> str:
        if self.hexagonal:
            return "hexagonal"
        if self.b is not None:
            return "orthorhombic"
        return "cubic" if self.c is None else "tetragonal"

    def cell_kwargs(self) -> dict:
        return {"a": self.a, "b": self.b, "c": self.c, "hexagonal": self.hexagonal}

    @property
    def volume(self) -> float:
        """Conventional-cell volume in cubic angstrom."""
        if self.hexagonal:
            return self.a * self.a * self.c * 0.8660254037844386
        if self.b is None and self.c is None:
            return self.a ** 3
        return self.a * (self.b or self.a) * self.c


def _fcc(element: str, b_iso: float = 0.5) -> tuple[Site, ...]:
    return tuple(Site(element, x, y, z, b_iso=b_iso) for x, y, z in
                 [(0, 0, 0), (0, .5, .5), (.5, 0, .5), (.5, .5, 0)])


def _rocksalt(cation: str, anion: str) -> tuple[Site, ...]:
    sites = list(_fcc(cation))
    sites += [Site(anion, x + .5, y, z) for x, y, z in
              [(0, 0, 0), (0, .5, .5), (.5, 0, .5), (.5, .5, 0)]]
    return tuple(sites)


PHASES: dict[str, Phase] = {
    "si": Phase(
        key="si", name="Silicon", formula="Si", space_group="Fd-3m (227)",
        a=5.431144, sites=tuple(
            [Site("Si", x, y, z) for x, y, z in
             [(0, 0, 0), (0, .5, .5), (.5, 0, .5), (.5, .5, 0),
              (.25, .25, .25), (.25, .75, .75), (.75, .25, .75), (.75, .75, .25)]]),
        source="NIST SRM 640f certificate, a = 5.431144(8) A at 22.5 C"),
    "al": Phase(
        key="al", name="Aluminium", formula="Al", space_group="Fm-3m (225)",
        a=4.0495, sites=_fcc("Al"),
        source="room-temperature conventional lattice parameter, a = 4.0495 A"),
    "cu": Phase(
        key="cu", name="Copper", formula="Cu", space_group="Fm-3m (225)",
        a=3.6149, sites=_fcc("Cu"),
        source="room-temperature conventional lattice parameter, a = 3.6149 A"),
    "fe": Phase(
        key="fe", name="alpha-Iron", formula="Fe", space_group="Im-3m (229)",
        a=2.8665, sites=(Site("Fe", 0, 0, 0), Site("Fe", .5, .5, .5)),
        source="room-temperature conventional lattice parameter, a = 2.8665 A"),
    "nacl": Phase(
        key="nacl", name="Rock salt", formula="NaCl", space_group="Fm-3m (225)",
        a=5.6402, sites=_rocksalt("Na", "Cl"),
        source="room-temperature conventional lattice parameter, a = 5.6402 A"),
    "mgo": Phase(
        key="mgo", name="Periclase", formula="MgO", space_group="Fm-3m (225)",
        a=4.2112, sites=_rocksalt("Mg", "O"),
        source="room-temperature conventional lattice parameter, a = 4.2112 A"),
    "zno": Phase(
        key="zno", name="Zincite", formula="ZnO", space_group="P63mc (186)",
        a=3.2494, c=5.2038, hexagonal=True, sites=(
            Site("Zn", 1/3, 2/3, 0.0), Site("Zn", 2/3, 1/3, 0.5),
            Site("O", 1/3, 2/3, 0.3830), Site("O", 2/3, 1/3, 0.8830)),
        source="COD 9004180, a = 3.2494, c = 5.2038 A; u(O) = 0.3830"),
    "tio2-rutile": Phase(
        key="tio2-rutile", name="Rutile", formula="TiO2", space_group="P42/mnm (136)",
        a=4.5937, c=2.9587, sites=(
            Site("Ti", 0, 0, 0), Site("Ti", .5, .5, .5),
            Site("O", .3047, .3047, 0), Site("O", .6953, .6953, 0),
```

```
            Site("O", .8047, .1953, .5), Site("O", .1953, .8047, .5)),
        source="room-temperature conventional cell, a = 4.5937, c = 2.9587 A; u(O) = 0.3047"),
    "tio2-anatase": Phase(
        key="tio2-anatase", name="Anatase", formula="TiO2", space_group="I41/amd (141)",
        a=3.7845, c=9.5143, sites=(
            Site("Ti", 0, 0, 0), Site("Ti", 0, .5, .25),
            Site("Ti", .5, .5, .5), Site("Ti", .5, 0, .75),
            Site("O", 0, 0, .2079), Site("O", 0, .5, .4579),
            Site("O", .5, .5, .7079), Site("O", .5, 0, .9579),
            Site("O", 0, .5, .0421), Site("O", 0, 0, .7921),
            Site("O", .5, 0, .5421), Site("O", .5, .5, .2921)),
        source="room-temperature conventional cell, a = 3.7845, c = 9.5143 A; z(O) = 0.2079"),
}


def get_phase(key: str) -> Phase:
    aliases = {"bcc": "fe", "fcc": "cu"}
    key = aliases.get(key, key)
    if key == "sc":
        return simple_cubic()
    try:
        return PHASES[key]
    except KeyError:
        choices = ", ".join(sorted(list(PHASES) + ["sc", "bcc", "fcc", "amorphous"]))
        raise KeyError(f"unknown phase {key!r}; choose from {choices}") from None


def simple_cubic(a: float = 3.359, element: str = "Cu") -> Phase:
    """Generic simple-cubic (Pm-3m) 1-atom model at cell edge `a` (default 3.359 A)."""
    return Phase(
        key="sc", name="Simple cubic", formula=f"{element} (SC)",
        space_group="Pm-3m (221)", a=a, sites=(Site(element, 0, 0, 0),),
        source=f"generic simple-cubic 1-atom model (Pm-3m), a = {a:.4f} A",
    )
```

xrdsim/sources.py

```
"""X-ray source: the copper tube and its emission lines.

Wavelengths in angstrom. The K-alpha doublet is what a powder pattern shows; K-beta is
normally removed by a filter or a monochromator and is only modelled to show why.
"""

from __future__ import annotations

from dataclasses import dataclass

CU_KA1 = 1.540598
CU_KA2 = 1.544426
CU_KB = 1.392250

# K-alpha2 / K-alpha1 intensity ratio, and K-beta / K-alpha ratio for a Cu tube.
KA2_TO_KA1 = 0.497
KB_TO_KA = 0.20


@dataclass(frozen=True)
class Line:
    wavelength: float
    weight: float


def emission_lines(ka2: bool = True, kbeta: bool = False) -> list[Line]:
    """Emission lines of a Cu tube with their relative intensities."""
    lines = [Line(CU_KA1, 1.0)]
    if ka2:
        lines.append(Line(CU_KA2, KA2_TO_KA1))
    if kbeta:
        lines.append(Line(CU_KB, KB_TO_KA * (1.0 + (KA2_TO_KA1 if ka2 else 0.0))))
    return lines


def weighted_wavelength(lines: list[Line]) -> float:
    """Intensity-weighted mean wavelength of the lines in use."""
    total = sum(line.weight for line in lines)
    if total <= 0:
        raise ValueError("no lines")
    return sum(line.wavelength * line.weight for line in lines) / total
```

xrdsim/structure.py

```
"""Structure factors, multiplicities and systematic absences.
A phase carries its cell and an explicit, symmetry-expanded basis; this module turns
```

```
that into |F(hkl)|^2 and the multiplicity the powder average needs.
"""

from __future__ import annotations

import cmath
import math

from .atoms import debye_waller, f0
from .lattice import d_spacing


def structure_factor(phase, h: int, k: int, l: int, wavelength: float) -> complex:
    """F(hkl) in electrons, including the Debye-Waller term of every site."""
    d = d_spacing(h, k, l, **phase.cell_kwargs())
    s = 1.0 / (2.0 * d)                 # sin(theta)/lambda, by Bragg's law
    total = 0j
    for site in phase.sites:
        phase_angle = 2.0 * math.pi * (h * site.x + k * site.y + l * site.z)
        total += (site.occupancy * f0(site.element, s) * debye_waller(site.b_iso, s)
                  * cmath.exp(1j * phase_angle))
    return total


def multiplicity(h: int, k: int, l: int, system: str) -> int:
    """Number of symmetry-equivalent planes in the powder average."""
    h, k, l = abs(h), abs(k), abs(l)
    if system == "cubic":
        perms = 6 if len({h, k, l}) == 3 else (3 if len({h, k, l}) == 2 else 1)
        return perms * 2 ** sum(v != 0 for v in (h, k, l))
    if system == "tetragonal":
        hk = 2 if h != k else 1
        return (hk * 2 ** sum(v != 0 for v in (h, k))) * (2 if l else 1)
    if system == "hexagonal":
        if h == 0 and k == 0:
            return 2 if l else 1
        # (h, k) on a mirror plane (h == 0, k == 0 or h == k) has a 6-member in-plane
        # orbit; a general (h, k) has 12. A non-zero l doubles either.
        in_plane = 6 if (h == 0 or k == 0 or h == k) else 12
        return 2 * in_plane if l else in_plane
    raise ValueError(f"multiplicity is not defined for {system!r}")


def lorentz_polarisation(
    two_theta_deg: float, monochromator_two_theta: float | None = None
) -> float:
    """LP factor for an unpolarised tube, optionally with a monochromator."""
    t = math.radians(two_theta_deg)
    theta = t / 2.0
    cos2t = math.cos(t) ** 2
    if monochromator_two_theta is None:
        numerator = 1.0 + cos2t
    else:
        numerator = 1.0 + cos2t * math.cos(math.radians(monochromator_two_theta)) ** 2
    return numerator / (math.sin(theta) ** 2 * math.cos(theta))


def relative_intensity(phase, h: int, k: int, l: int, wavelength: float,
                       monochromator_two_theta: float | None = None) -> float:
    """I/I0 for one reflection: |F|^2 m LP, normalised later across the pattern."""
    f = structure_factor(phase, h, k, l, wavelength)
    d = d_spacing(h, k, l, **phase.cell_kwargs())
    two_theta = math.degrees(2.0 * math.asin(wavelength / (2.0 * d)))
    m = multiplicity(h, k, l, phase.system)
    return abs(f) ** 2 * m * lorentz_polarisation(two_theta, monochromator_two_theta)
```

report/size_study.py

```
"""Crystallite-size study for items 2 and 3: the numbers the report quotes.

Writes report/size_table.md and returns the same rows to the report builder.
"""

from __future__ import annotations

import pathlib

from xrdsim.analysis import find_peaks, scherrer_from_peak
from xrdsim.diffractometer import Instrument, scan
from xrdsim.profile import caglioti_fwhm, size_fwhm
from xrdsim.samples import get_phase

HERE = pathlib.Path(__file__).resolve().parent
```

```
TABLE = HERE / "size_table.md"

SIZES_A = [50.0, 100.0, 200.0, 500.0, 1000.0, 2000.0, 5000.0]   # 5 nm to 500 nm
HKLS = [(1, 1, 1), (2, 2, 0), (4, 0, 0)]
STEP = 0.002
BASE = Instrument(
    reference_counts=100_000.0, background_rate=0.0, ka2=False, rng_seed=None
)


def rows() -> list[dict]:
    si = get_phase("si")
    out = []
    for size in SIZES_A:
        result = scan(si, 12.0, 75.0, STEP, BASE, size_a=size)
        peaks = find_peaks(result.two_theta, result.counts)
        by_angle = {round(peak.two_theta, 1): peak for peak in peaks}
        for hkl in HKLS:
            predicted = next(row for row in result.reflections if row.hkl == hkl)
            peak = min(peaks, key=lambda p: abs(p.two_theta - predicted.two_theta))
            instrumental = caglioti_fwhm(peak.two_theta, *BASE.caglioti)
            predicted_size = size_fwhm(size, peak.two_theta, BASE.wavelength)
            try:
                recovered = scherrer_from_peak(peak, instrumental, BASE.wavelength) / 10.0
            except ValueError:
                recovered = float("nan")
            out.append({
                "size_nm": size / 10.0,
                "hkl": "".join(str(v) for v in hkl),
                "two_theta": peak.two_theta,
                "fwhm": peak.fwhm,
                "instrument_fwhm": instrumental,
                "predicted_size_fwhm": predicted_size,
                "height": peak.height,
                "area": peak.area,
                "recovered_nm": recovered,
            })
    return out


def build() -> str:
    data = rows()
    lines = [
        "# Crystallite-size study",
        "",
        "Silicon, Cu K-alpha1 only (the doublet would confuse a width measurement), "
        f"step {STEP} deg, 100 000 counts at the strongest line, no counting noise.",
        "",
        (
            "| D (nm) | hkl | 2theta (deg) | FWHM (deg) | instrument FWHM "
            "| height (counts) | area (counts deg) | Scherrer (nm) |"
        ),
        "| --- | --- | --- | --- | --- | --- | --- | --- |",
    ]
    for row in data:
        lines.append(
            f"| {row['size_nm']:.0f} | {row['hkl']} | {row['two_theta']:.3f} | "
            f"{row['fwhm']:.4f} | {row['instrument_fwhm']:.4f} | {row['height']:.0f} | "
            f"{row['area']:.0f} | {row['recovered_nm']:.0f} |")
    text = "\n".join(lines) + "\n"
    TABLE.write_text(text, encoding="utf-8")
    print(f"wrote {TABLE} ({len(data)} rows)")
    return text


if __name__ == "__main__":
    build()
```

report/tables.py

```
"""Write report/tables.md: the reflection tables the report quotes.

Generated, never edited by hand, so a number in the report can always be traced to
the model that produced it.
"""

from __future__ import annotations

import pathlib

from xrdsim.diffractometer import Instrument, reflections
from xrdsim.samples import PHASES
HERE = pathlib.Path(__file__).resolve().parent
```

```
OUT = HERE / "tables.md"


def build() -> None:
    instrument = Instrument()
    lines = [
        "# Reflection tables",
        "",
        "Generated by `uv run python -m report.tables`. Cu K-alpha1 = "
        f"{instrument.wavelength:.6f} A, K-alpha2 included in the scan but not in the "
        "intensity column, which is relative to the strongest reflection of each phase.",
        "",
    ]
    for key, phase in PHASES.items():
        rows = [row for row in reflections(phase, instrument) if row.intensity > 1e-9]
        if not rows:
            continue
        strongest = max(row.intensity for row in rows)
        lines += [
            f"## {phase.name} ({phase.formula}), {phase.space_group}",
            "",
            f"Cell: a = {phase.a:.4f}" + (f", c = {phase.c:.4f}" if phase.c else "")
            + f" A. Source: {phase.source}",
            "",
            "| hkl | d (A) | 2theta (deg) | m | I/I0 (%) |",
            "| --- | --- | --- | --- | --- |",
        ]
        for row in rows:
            label = "".join(str(v) for v in row.hkl)
            lines.append(f"| {label} | {row.d:.4f} | {row.two_theta:.3f} | "
                         f"{row.multiplicity} | {100.0 * row.intensity / strongest:.1f} |")
        lines.append("")
    OUT.write_text("\n".join(lines), encoding="utf-8")
    print(f"wrote {OUT} ({len(lines)} lines)")


if __name__ == "__main__":
    build()
```

report/make_figures.py

```
"""Figures for the assignment 2 report.

Every figure here is written straight from the simulator, so a figure cannot drift from
the model. Run it with:

    uv run python -m report.make_figures
"""

from __future__ import annotations

import contextlib
import io
import pathlib

import matplotlib
import numpy as np

matplotlib.use("Agg")

import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, Rectangle

from xrdsim.diffractometer import Instrument, scan
from xrdsim.plot import plot_scan
from xrdsim.samples import PHASES, get_phase

HERE = pathlib.Path(__file__).resolve().parent
OUT = HERE / "figures"
LAMBDA = 1.540598


def instrument_diagram() -> None:
    """Block diagram of the Bragg-Brentano chain."""
    fig, ax = plt.subplots(figsize=(11, 2.6))
    boxes = [
        ("X-ray tube\nCu K-alpha", 0.5), ("divergence\nslit", 2.1),
        ("Soller slit", 3.4), ("sample on\ntheta/2theta", 4.9),
        ("receiving\nslit", 6.4), ("monochromator\n(optional)", 7.9),
        ("detector", 9.4),
    ]
    for label, x in boxes:
        rect = Rectangle((x, 0.40), 1.1, 0.42, fill=False, lw=1.2, color="#1a3d6d")
```

```
        ax.add_patch(rect)
        ax.text(x + 0.55, 0.61, label, ha="center", va="center", fontsize=8)
    for (_, x1), (_, x2) in zip(boxes, boxes[1:]):
        ax.add_patch(FancyArrowPatch((x1 + 0.55, 0.61), (x2 + 0.05, 0.61),
                                     arrowstyle="-|>", mutation_scale=10, lw=1.0,
                                     color="#8a3b12"))
    ax.annotate("2theta scan", xy=(4.9 + 0.55, 0.40), xytext=(4.9 + 0.55, 0.16),
                ha="center", fontsize=8, color="#8a3b12",
                arrowprops=dict(arrowstyle="-|>", lw=1.0, color="#8a3b12"))
    ax.set_xlim(0, 11)
    ax.set_ylim(0, 1.0)
    ax.axis("off")
    fig.tight_layout()
    fig.savefig(OUT / "instrument.png", dpi=160)
    plt.close(fig)


def phase_panel() -> None:
    """One pattern per phase, on a common 2theta axis."""
    keys = [
        "si", "al", "cu", "fe", "nacl",
        "mgo", "zno", "tio2-rutile", "tio2-anatase",
    ]
    fig, axes = plt.subplots(3, 3, figsize=(10.5, 10.5), sharex=True)
    inst = Instrument(background_rate=0.0, rng_seed=None)
    for ax, key in zip(axes.ravel(), keys):
        phase = get_phase(key)
        result = scan(phase, 20.0, 90.0, 0.01, inst)
        plot_scan(result, ax=ax, title=f"{phase.name} ({phase.formula})", annotate=True)
        ax.set_ylabel("counts")
        ax.tick_params(labelsize=8)
    fig.tight_layout()
    fig.savefig(OUT / "phase_panel.png", dpi=150)
    plt.close(fig)


def scherrer_demo() -> None:
    """The same phase at three crystallite sizes, on the 111 line."""
    fig, ax = plt.subplots(figsize=(8, 4))
    inst = Instrument(ka2=False, background_rate=0.0, rng_seed=None)
    for size, colour in [(50.0, "#8a3b12"), (200.0, "#1a3d6d"), (2000.0, "#2f7d32")]:
        result = scan(get_phase("si"), 25.5, 31.0, 0.005, inst, size_a=size)
        ax.plot(result.two_theta, result.counts / result.counts.max(),
                lw=1.2, color=colour, label=f"D = {size / 10:.0f} nm")
    ax.set_xlabel("2theta (deg)")
    ax.set_ylabel("normalised counts")
    ax.set_title(
        "Silicon 111: instrumental broadening plus crystallite size (K-alpha1 only)"
    )
    ax.legend(frameon=False)
    ax.grid(alpha=0.25, lw=0.5)
    fig.tight_layout()
    fig.savefig(OUT / "scherrer.png", dpi=160)
    plt.close(fig)


def doublet_demo() -> None:
    """The K-alpha doublet, resolved and unresolved."""
    fig, ax = plt.subplots(figsize=(8, 4))
    for ka2, label, colour in [(False, "K-alpha1 only", "#1a3d6d"),
                               (True, "K-alpha1 + K-alpha2", "#8a3b12")]:
        inst = Instrument(ka2=ka2, background_rate=0.0, rng_seed=None)
        result = scan(get_phase("si"), 87.0, 89.0, 0.002, inst)
        ax.plot(result.two_theta, result.counts / result.counts.max(),
                lw=1.2, color=colour, label=label)
    ax.set_xlabel("2theta (deg)")
    ax.set_ylabel("normalised counts")
    ax.set_title("Silicon 422 at 88 deg: the doublet is 0.28 deg wide")
    ax.legend(frameon=False)
    ax.grid(alpha=0.25, lw=0.5)
    fig.tight_layout()
    fig.savefig(OUT / "doublet.png", dpi=160)
    plt.close(fig)


def size_series() -> None:
    """R1: the same phase at the seven crystallite sizes of Table 4, raw and normalised."""
    sizes = [
        (50.0, "5 nm"), (100.0, "10 nm"), (200.0, "20 nm"),
        (500.0, "50 nm"), (1000.0, "100 nm"), (2000.0, "200 nm"), (5000.0, "500 nm"),
    ]
    fig, (left, right) = plt.subplots(1, 2, figsize=(11, 4.2), sharex=True)
    colours = [
        "#8a3b12", "#b5651d", "#d9822b", "#7a8b3a", "#2f7d32", "#1f6f8b", "#1a3d6d",
```

```
    ]
    inst = Instrument(ka2=False, background_rate=0.0, rng_seed=None)
    for (size, label), colour in zip(sizes, colours):
        result = scan(get_phase("si"), 27.4, 29.4, 0.002, inst, size_a=size)
        left.plot(result.two_theta, result.counts, lw=1.1, color=colour, label=label)
        right.plot(result.two_theta, result.counts / result.counts.max(), lw=1.1,
                   color=colour, label=label)
    left.set_title("same reflection, counts as measured")
    right.set_title("same curves, each normalised to its own peak")
    for ax in (left, right):
        ax.set_xlabel("2theta (deg)")
        ax.grid(alpha=0.25, lw=0.5)
    left.set_ylabel("counts")
    right.set_ylabel("normalised counts")
    right.legend(frameon=False, fontsize=8, title="crystallite size")
    fig.suptitle(
        "Silicon 111: crystallite size changes the width, not the position", y=1.0
    )
    fig.tight_layout()
    fig.savefig(OUT / "size_series.png", dpi=160, bbox_inches="tight")
    plt.close(fig)


def size_laws() -> None:
    """R2: FWHM against 1/D, and peak height against D."""
    from .size_study import rows as size_rows
    data = size_rows()
    fig, (left, right) = plt.subplots(1, 2, figsize=(11, 4.0))
    for hkl, colour in [("111", "#1a3d6d"), ("220", "#8a3b12"), ("400", "#2f7d32")]:
        subset = [row for row in data if row["hkl"] == hkl]
        sizes = [row["size_nm"] for row in subset]
        widths = [row["fwhm"] for row in subset]
        left.loglog(1.0 / np.array(sizes), widths, "o-", ms=4, lw=1.1,
                    color=colour, label=f"{hkl}")
        heights = [row["height"] for row in subset]
        right.loglog(sizes, heights, "o-", ms=4, lw=1.1, color=colour, label=f"{hkl}")
    left.set_xlabel("1 / D (1/nm)")
    left.set_ylabel("measured FWHM (deg)")
    left.set_title("width against inverse size")
    left.legend(frameon=False, fontsize=8)
    left.grid(alpha=0.25, lw=0.5, which="both")
    right.set_xlabel("D (nm)")
    right.set_ylabel("peak height (counts)")
    right.set_title("height against size")
    right.legend(frameon=False, fontsize=8)
    right.grid(alpha=0.25, lw=0.5, which="both")
    fig.tight_layout()
    fig.savefig(OUT / "size_laws.png", dpi=160)
    plt.close(fig)


def fwhm_vs_angle() -> None:
    """R2: the same size broadens by 1/cos(theta), so width grows with angle."""
    from xrdsim.analysis import find_peaks
    fig, ax = plt.subplots(figsize=(8, 4))
    inst = Instrument(ka2=False, background_rate=0.0, rng_seed=None)
    for size, colour in [(200.0, "#1a3d6d"), (500.0, "#8a3b12")]:
        result = scan(get_phase("si"), 20.0, 120.0, 0.005, inst, size_a=size)
        peaks = [peak for peak in find_peaks(result.two_theta, result.counts)
                 if peak.fwhm > 0 and peak.height > 0.02 * result.counts.max()]
        ax.plot([peak.two_theta for peak in peaks], [peak.fwhm for peak in peaks],
                "o", ms=4, color=colour, label=f"D = {size / 10:.0f} nm")
    angles = np.linspace(22.0, 118.0, 200)
    cos_th = np.cos(np.radians(angles / 2.0))
    scale = 0.9 * 1.540598 / (200.0 * cos_th) * 180 / np.pi
    ax.plot(angles, scale, lw=1.0, ls="--", color="#666666",
            label="0.9 lambda / (D cos theta), D = 20 nm")
    ax.set_xlabel("2theta (deg)")
    ax.set_ylabel("FWHM (deg)")
    ax.set_yscale("log")
    ax.set_title("the same crystallite broadens more at high angle")
    ax.legend(frameon=False, fontsize=8)
    ax.grid(alpha=0.25, lw=0.5, which="both")
    fig.tight_layout()
    fig.savefig(OUT / "fwhm_vs_angle.png", dpi=160)
    plt.close(fig)


def convolution_demo() -> None:
    """R3: ideal comb -> exact Laue sinc^2 -> numerical convolution -> inst-limited."""
    import math
    from xrdsim.profile import (add_in_quadrature, caglioti_fwhm,
                                convolve_laue_with_instrument,
                                laue_interference_array, pseudo_voigt_array,
```

```
                            size_fwhm)
fig, axes = plt.subplots(4, 1, figsize=(9.2, 8.4), sharex=True)
x = np.linspace(27.5, 29.8, 4601)
tth_b = 28.441
d_111 = 5.431144 / math.sqrt(3.0)
inst_w = caglioti_fwhm(tth_b, 0.004, -0.002, 0.003)

comb = np.zeros_like(x)
comb[np.argmin(np.abs(x - tth_b))] = 1.0
axes[0].plot(x, comb, lw=1.2, color="#1a3d6d",
             label="Dirac impulse at 2theta_B = 28.441 deg")
axes[0].set_ylim(-0.05, 1.25)
axes[0].set_title(
    "(a) Ideal infinite lattice: Dirac impulse train at the Bragg angle",
    fontsize=9.5,
)
axes[0].legend(loc="upper right", frameon=False, fontsize=8)

n_mid = 160  # D = 160 * 0.31357 nm = 50.2 nm
laue_mid = laue_interference_array(x, tth_b, d_111, n_mid, LAMBDA)
axes[1].plot(
    x, laue_mid, lw=1.3, color="#8a3b12",
    label=f"Exact Laue sinc^2 (N = {n_mid} planes, D = 50.2 nm, FWHM = 0.1608 deg)",
)
zoom_curve = np.clip(laue_mid * 8.0, 0, laue_mid.max() * 0.85)
axes[1].plot(x, zoom_curve, lw=0.9, ls="--", color="#d95f02",
             label="8x vertical zoom showing sinc^2 secondary sidelobes")
axes[1].set_ylim(-0.2, laue_mid.max() * 1.25)
axes[1].set_title(
    "(b) Spatial window w(r): exact finite-chain Laue sinc^2 response |W(q)|^2",
    fontsize=9.5,
)
axes[1].legend(loc="upper right", frameon=False, fontsize=8)

conv_mid = convolve_laue_with_instrument(x, tth_b, d_111, n_mid, LAMBDA, inst_w)
size_w_mid = size_fwhm(n_mid * d_111, tth_b, LAMBDA, k=0.8859)
pv_w_mid = add_in_quadrature(size_w_mid, inst_w)
pv_mid = pseudo_voigt_array(x, tth_b, pv_w_mid, 0.5)
sigma_inst = inst_w / (2.0 * math.sqrt(2.0 * math.log(2.0)))
h_inst_curve = (
    np.exp(-0.5 * ((x - tth_b) / sigma_inst) ** 2)
    / (sigma_inst * math.sqrt(2.0 * math.pi))
)
axes[2].plot(x, laue_mid, lw=0.9, ls="--", color="#8a3b12", alpha=0.65,
             label="Pure Laue |W(q)|^2 before instrument (FWHM = 0.1608 deg)")
axes[2].plot(x, h_inst_curve * 0.18, lw=0.9, ls="-.", color="#666666",
             label="Gaussian instrument kernel h_inst (FWHM = 0.0524 deg, x0.18)")
axes[2].plot(x, conv_mid, lw=1.4, color="#2f7d32",
             label="Numerical convolution |W|^2 * h_inst (FWHM = 0.1671 deg)")
axes[2].plot(
    x, pv_mid, lw=1.1, ls=":", color="#1a3d6d",
    label="Analytical pseudo-Voigt quadrature (K = 0.8859, FWHM = 0.1691 deg)",
)
axes[2].set_ylim(-0.2, laue_mid.max() * 1.28)
axes[2].set_title(
    "(c) Second LTI stage: numerical convolution |W|^2 * h_inst damps sidelobes",
    fontsize=9.5,
)
axes[2].legend(loc="upper right", frameon=False, fontsize=7.6)

n_big = 1600  # D = 501.7 nm
laue_big = laue_interference_array(x, tth_b, d_111, n_big, LAMBDA)
conv_big = convolve_laue_with_instrument(x, tth_b, d_111, n_big, LAMBDA, inst_w)
size_w_big = size_fwhm(n_big * d_111, tth_b, LAMBDA, k=0.8859)
pv_w_big = add_in_quadrature(size_w_big, inst_w)
pv_big = pseudo_voigt_array(x, tth_b, pv_w_big, 0.5)
axes[3].plot(
    x, laue_big * 0.35, lw=0.9, ls="--", color="#666666",
    label="Pure Laue |W(q)|^2 (N = 1600, D = 501.7 nm, FWHM = 0.0161 deg, x0.35)",
)
axes[3].plot(x, conv_big, lw=1.4, color="#6a3d9a",
             label="Numerical convolution |W|^2 * h_inst (FWHM = 0.0561 deg)")
axes[3].plot(
    x, pv_big, lw=1.1, ls=":", color="#1a3d6d",
    label="Analytical pseudo-Voigt quadrature (K = 0.8859, FWHM = 0.0549 deg)",
)
axes[3].set_ylim(-0.8, (laue_big.max() * 0.35) * 1.25)
axes[3].set_title(
    "(d) Instrument-limited regime (D = 501.7 nm): h_inst sets the observed width",
    fontsize=9.5,
)
axes[3].legend(loc="upper right", frameon=False, fontsize=7.6)

for ax in axes:
```

```
        ax.set_ylabel("density")
        ax.grid(alpha=0.25, lw=0.5)
    axes[-1].set_xlabel("2theta (deg)")
    fig.tight_layout()
    fig.savefig(OUT / "convolution.png", dpi=160)
    plt.close(fig)


def laue_plot() -> None:
    """R3: the interference function of a finite chain, width proportional to 1/N."""
    v = np.linspace(-0.5, 0.5, 4000)
    fig, ax = plt.subplots(figsize=(8, 4))
    for planes, colour in [(5, "#8a3b12"), (20, "#1a3d6d"), (100, "#2f7d32")]:
        y = (np.sin(planes * np.pi * v) / np.sin(np.pi * v)) ** 2
        ax.plot(v, y / planes ** 2, lw=1.1, color=colour, label=f"N = {planes} planes")
    ax.set_xlabel("reduced reciprocal coordinate between Bragg points, q d / 2 pi")
    ax.set_ylabel("normalised intensity")
    ax.set_title(
        "Finite-chain interference |sin(N pi v)/sin(pi v)|^2: "
        "width ~ 1/N, powder area conserved",
        fontsize=9.5,
    )
    ax.legend(frameon=False, fontsize=8)
    ax.grid(alpha=0.25, lw=0.5)
    fig.tight_layout()
    fig.savefig(OUT / "laue.png", dpi=160)
    plt.close(fig)


def class_spec_figure() -> None:
    """In-class assignment specification:
    1. Crystal types: Simple Cubic (SC), BCC, FCC, and Amorphous.
    2. Variable: Lattice constant a variation.
    3. Input: Cu K-alpha radiation (1.5406 A) and full-width CLI session capture.
    """
    from xrdsim.cli import main as cli_main
    from xrdsim.diffractometer import amorphous_scan
    from xrdsim.samples import Phase, Site, simple_cubic

    inst = Instrument(background_rate=0.0, rng_seed=None, ka2=False)

    fig = plt.figure(figsize=(11.6, 8.6), constrained_layout=True)
    gs = fig.add_gridspec(2, 3, height_ratios=[1.05, 1.0])

    ax_lattices = fig.add_subplot(gs[0, 0])
    sc_scan = scan(simple_cubic(a=3.359, element="Cu"), 20.0, 95.0, 0.02, inst)
    bcc_scan = scan(get_phase("fe"), 20.0, 95.0, 0.02, inst)
    fcc_scan = scan(get_phase("cu"), 20.0, 95.0, 0.02, inst)

    ax_lattices.plot(
        sc_scan.two_theta, sc_scan.counts / sc_scan.counts.max() + 2.1,
        lw=1.1, color="#1a3d6d",
        label="Generic SC (a = 3.359 \u00c5): all hkl",
    )
    ax_lattices.plot(
        bcc_scan.two_theta, bcc_scan.counts / bcc_scan.counts.max() + 1.05,
        lw=1.1, color="#8a3b12",
        label="BCC (a = 2.867 \u00c5): h+k+l = even",
    )
    ax_lattices.plot(
        fcc_scan.two_theta, fcc_scan.counts / fcc_scan.counts.max(),
        lw=1.1, color="#2f7d32",
        label="FCC (a = 3.615 \u00c5): unmixed parity",
    )
    ax_lattices.set_title(
        "(a) Crystal Types: SC vs. BCC vs. FCC",
        fontsize=9.2, fontweight="bold",
    )
    ax_lattices.set_xlabel("2\u03b8 (\u00b0)", fontsize=8.5)
    ax_lattices.set_ylabel("Normalized Intensity (stacked)", fontsize=8.5)
    ax_lattices.set_xlim(20, 95)
    ax_lattices.set_ylim(-0.08, 3.85)
    ax_lattices.legend(loc="upper right", frameon=True, fontsize=7.2)
    ax_lattices.grid(alpha=0.2, lw=0.5)

    ax_amorph = fig.add_subplot(gs[0, 1])
    am_scan = amorphous_scan(10.0, 90.0, 0.02, inst, center_deg=21.8, fwhm_deg=8.8)
    ax_amorph.plot(
        am_scan.two_theta, am_scan.counts / am_scan.counts.max(),
        lw=1.8, color="#d95f02",
        label="Amorphous SiO2 (halo at 21.8\u00b0, FWHM = 8.8\u00b0)",
    )
    si_scan = scan(get_phase("si"), 10.0, 90.0, 0.02, inst)
    ax_amorph.plot(
```

```
        si_scan.two_theta, si_scan.counts / si_scan.counts.max(),
        lw=1.1, color="#1a3d6d",
        label="Crystalline Si (sharp Bragg peaks)",
    )
    ax_amorph.set_title(
        "(b) Crystalline vs. Amorphous Phase",
        fontsize=9.2, fontweight="bold",
    )
    ax_amorph.set_xlabel("2\u03b8 (\u00b0)", fontsize=8.5)
    ax_amorph.set_ylabel("Normalized Intensity", fontsize=8.5)
    ax_amorph.set_xlim(10, 90)
    ax_amorph.set_ylim(-0.04, 1.32)
    ax_amorph.legend(loc="upper right", frameon=True, fontsize=7.2)
    ax_amorph.grid(alpha=0.2, lw=0.5)

    ax_param = fig.add_subplot(gs[0, 2])
    colors_a = ["#4575b4", "#74add1", "#f46d43", "#d73027"]
    a_values = [3.20, 3.80, 4.40, 5.00]
    fcc_sites = (
        Site("Cu", 0, 0, 0), Site("Cu", 0, 0.5, 0.5),
        Site("Cu", 0.5, 0, 0.5), Site("Cu", 0.5, 0.5, 0),
    )
    for a_val, col in zip(a_values, colors_a):
        phase_a = Phase(
            key=f"fcc_{a_val}", name=f"FCC (a = {a_val:.2f} \u00c5)",
            formula="M", space_group="Fm-3m", a=a_val, sites=fcc_sites,
        )
        res_a = scan(phase_a, 20.0, 85.0, 0.02, inst)
        ax_param.plot(res_a.two_theta, res_a.counts / res_a.counts.max(),
                      lw=1.2, color=col, label=f"--a {a_val:.2f} \u00c5")

    ax_param.set_title(
        "(c) Varying Lattice Constant a (--a)",
        fontsize=9.2, fontweight="bold",
    )
    ax_param.set_xlabel("2\u03b8 (\u00b0)", fontsize=8.5)
    ax_param.set_ylabel("Normalized Intensity", fontsize=8.5)
    ax_param.set_xlim(20, 85)
    ax_param.set_ylim(-0.04, 1.28)
    ax_param.legend(loc="upper right", frameon=True, fontsize=7.5)
    ax_param.grid(alpha=0.2, lw=0.5)

    ax_cli = fig.add_subplot(gs[1, :])
    shot_path = OUT / "terminal_screenshot.png"
    if shot_path.exists():
        ax_cli.imshow(plt.imread(shot_path), interpolation="lanczos")
        ax_cli.axis("off")
    else:
        buf = io.StringIO()
        with contextlib.redirect_stdout(buf):
            cli_main(["--phase", "si", "--size-nm", "20", "--index", "--scherrer"])
        ax_cli.text(0.02, 0.95, buf.getvalue(), transform=ax_cli.transAxes,
                    fontfamily="monospace", fontsize=8.0, va="top")
    ax_cli.set_title(
        "(d) Windows Terminal Session Screenshot: SC (--a 3.359), "
        "Amorphous (--phase amorphous), and Si (--size-nm 20)",
        fontsize=9.2, fontweight="bold",
    )

    fig.savefig(OUT / "class_spec.png", dpi=180)
    plt.close(fig)


def main() -> None:
    OUT.mkdir(parents=True, exist_ok=True)
    instrument_diagram()
    class_spec_figure()
    phase_panel()
    scherrer_demo()
    doublet_demo()
    size_series()
    size_laws()
    fwhm_vs_angle()
    convolution_demo()
    laue_plot()
    print(f"wrote {len(list(OUT.glob('*.png')))} figures to {OUT}")


if __name__ == "__main__":
    main()
```

Appendix B: constants and their provenance

The cell constants the report uses, with the source behind each value [3], [12], [14].

Key	Cell (Å)	Source [3], [12], [14]
si	a = 5.4311	NIST SRM 640f certificate, a = 5.431144(8) A at 22.5 C
al	a = 4.0495	room-temperature conventional lattice parameter, a = 4.0495 A
cu	a = 3.6149	room-temperature conventional lattice parameter, a = 3.6149 A
fe	a = 2.8665	room-temperature conventional lattice parameter, a = 2.8665 A
nacl	a = 5.6402	room-temperature conventional lattice parameter, a = 5.6402 A
mgo	a = 4.2112	room-temperature conventional lattice parameter, a = 4.2112 A
zno	a = 3.2494, c = 5.2038	COD 9004180, a = 3.2494, c = 5.2038 A; u(O) = 0.3830
tio2-rutile	a = 4.5937, c = 2.9587	room-temperature conventional cell, a = 4.5937, c = 2.9587 A; u(O) = 0.3047
tio2-anatase	a = 3.7845, c = 9.5143	room-temperature conventional cell, a = 3.7845, c = 9.5143 A; z(O) = 0.2079

Table 9: cell constants and the source behind each [3], [12], [14].

Instrument constants: Cu Kα1 = 1.540598 Å, Kα2 = 1.544426 Å (ratio 0.497), Kβ = 1.392250 Å; Caglioti U = 0.004, V = −0.002, W = 0.003 deg²; η = 0.5; scan ceiling 150°; Debye-Waller B = 0.5 Å² for every site. The form factors are the Waasmaier-Kirfel parameterisation [6], tabulated in data/scattering_factors.json.

Appendix C: reflection tables for the nine reference phases

Every reflection the model reaches below 150° with a non-zero structure factor, with its spacing, its angle, its multiplicity and its intensity relative to the strongest line of that phase. Generated by report/tables.py; the same table is in report/tables.md.

Silicon (Si), Fd-3m (227)

hkl	d (Å)	2θ (°)	m	I/I0 (%)
111	3.1357	28.441	8	100.0
220	1.9202	47.301	12	64.3
311	1.6376	56.120	24	37.3
400	1.3578	69.127	6	9.6
331	1.2460	76.373	24	14.1
422	1.1086	88.026	24	19.4
333	1.0452	94.948	8	2.8
511	1.0452	94.948	24	8.3
440	0.9601	106.703	12	7.2
531	0.9180	114.086	48	14.0
620	0.8587	127.536	24	15.0
533	0.8282	136.883	24	8.5

Table 10: all non-zero reflections of Silicon below 150°.

Aluminium (Al), Fm-3m (225)

hkl	d (Å)	2θ (°)	m	I/I0 (%)
111	2.3380	38.473	8	100.0

hkl	d (Å)	2θ (°)	m	I/I0 (%)
200	2.0248	44.722	6	47.6
220	1.4317	65.099	12	28.5
311	1.2210	78.232	24	31.5
222	1.1690	82.439	8	9.0
400	1.0124	99.084	6	4.4
331	0.9290	112.024	24	15.6
420	0.9055	116.574	24	15.5
422	0.8266	137.462	24	18.4

Table 11: all non-zero reflections of Aluminium below 150°.

Copper (Cu), Fm-3m (225)

hkl	d (Å)	2θ (°)	m	I/I0 (%)
111	2.0871	43.318	8	100.0
200	1.8075	50.451	6	45.9
220	1.2781	74.128	12	24.2
311	1.0899	89.941	24	27.1
222	1.0435	95.151	8	8.0
400	0.9037	116.939	6	4.9
331	0.8293	136.509	24	23.5
420	0.8083	144.715	24	27.8

Table 12: all non-zero reflections of Copper below 150°.

alpha-Iron (Fe), Im-3m (229)

hkl	d (Å)	2θ (°)	m	I/I0 (%)
110	2.0269	44.672	12	100.0
200	1.4332	65.020	6	13.8
211	1.1702	82.331	24	24.8
220	1.0135	98.940	12	8.0
310	0.9065	116.376	24	14.2
222	0.8275	137.148	8	5.8

Table 13: all non-zero reflections of alpha-Iron below 150°.

Rock salt (NaCl), Fm-3m (225)

hkl	d (Å)	2θ (°)	m	I/I0 (%)
111	3.2564	27.366	8	8.2
200	2.8201	31.703	6	100.0
220	1.9941	45.447	12	63.8
311	1.7006	53.868	24	1.9
222	1.6282	56.472	8	19.9
400	1.4101	66.226	6	8.5
331	1.2940	73.069	24	0.9

hkl	d (Å)	2θ (°)	m	I/I0 (%)
420	1.2612	75.291	24	21.9
422	1.1513	83.990	24	15.7
333	1.0855	90.414	8	0.2
511	1.0855	90.414	24	0.7
440	0.9971	101.171	12	5.4
531	0.9534	107.797	48	1.5
442	0.9400	110.057	24	10.1
600	0.9400	110.057	6	2.5
620	0.8918	119.484	24	10.2
533	0.8601	127.163	24	1.0
622	0.8503	129.896	24	11.2
444	0.8141	142.243	8	4.6

Table 14: all non-zero reflections of Rock salt below 150°.

Periclase (MgO), Fm-3m (225)

hkl	d (Å)	2θ (°)	m	I/I0 (%)
111	2.4313	36.942	8	10.6
200	2.1056	42.918	6	100.0
220	1.4889	62.312	12	52.5
311	1.2697	74.698	24	6.3
222	1.2157	78.639	8	14.8
400	1.0528	94.053	6	6.5
331	0.9661	105.749	24	2.8
420	0.9417	109.776	24	20.4
422	0.8596	127.302	24	20.7
333	0.8104	143.781	8	1.1
511	0.8104	143.781	24	3.2

Table 15: all non-zero reflections of Periclase below 150°.

Zincite (ZnO), P63mc (186)

hkl	d (Å)	2θ (°)	m	I/I0 (%)
100	2.8141	31.773	6	52.1
002	2.6019	34.441	2	39.9
101	2.4753	36.262	12	100.0
102	1.9104	47.558	12	22.5
110	1.6247	56.604	6	33.3
103	1.4766	62.888	12	30.6
200	1.4070	66.386	6	4.7
112	1.3781	67.968	12	26.5
201	1.3583	69.100	12	13.5

hkl	d (Å)	2θ (°)	m	I/I0 (%)
004	1.3010	72.613	2	2.3
202	1.2377	76.981	12	4.4
104	1.1809	81.433	12	2.4
203	1.0927	89.647	12	9.7
210	1.0636	92.809	12	3.3
211	1.0421	95.327	24	10.9
114	1.0155	98.670	12	6.2
212	0.9845	102.962	24	4.3
105	0.9761	104.208	12	7.9
204	0.9552	107.494	12	1.4
300	0.9380	110.409	6	5.2
213	0.9067	116.322	24	13.9
302	0.8824	121.602	12	8.2
006	0.8673	125.285	2	1.2
205	0.8367	134.029	12	8.6
106	0.8288	136.678	12	2.1
214	0.8234	138.606	24	3.4
220	0.8123	142.968	6	7.0

Table 16: all non-zero reflections of Zincite below 150°.

Rutile (TiO2), P42/mnm (136)

hkl	d (Å)	2θ (°)	m	I/I0 (%)
110	3.2482	27.436	4	100.0
101	2.4874	36.080	8	45.4
200	2.2969	39.190	4	6.6
111	2.1873	41.239	8	18.6
210	2.0544	44.043	8	6.6
211	1.6875	54.321	16	55.0
220	1.6241	56.626	4	16.4
002	1.4793	62.759	2	7.7
310	1.4527	64.047	8	7.8
221	1.4237	65.510	8	0.6
301	1.3599	69.004	8	19.2
112	1.3463	69.802	8	9.4
311	1.3040	72.418	16	1.1
320	1.2741	74.400	8	0.2
202	1.2437	76.538	8	2.0
212	1.2005	79.831	16	1.1
321	1.1702	82.337	16	4.0

hkl	d (Å)	2θ (°)	m	I/I0 (%)
400	1.1484	84.249	4	2.8
410	1.1141	87.480	8	1.0
222	1.0937	89.551	8	6.7
330	1.0827	90.703	4	3.6
411	1.0427	95.255	16	6.3
312	1.0365	96.006	16	4.8
420	1.0272	97.166	8	2.8
331	1.0168	98.501	8	0.1
421	0.9704	105.088	16	0.6
322	0.9654	105.863	16	0.2
103	0.9643	106.041	8	2.0
113	0.9437	109.424	8	0.6
430	0.9187	113.949	8	0.2
402	0.9072	116.235	8	3.5
510	0.9009	117.527	8	4.0
412	0.8900	119.887	16	1.2
213	0.8891	120.084	16	7.6
431	0.8774	122.785	16	7.1
501	0.8774	122.785	8	0.8
332	0.8737	123.678	8	5.7
511	0.8618	126.708	16	0.0
520	0.8530	129.115	8	0.0
422	0.8437	131.836	16	5.8
223	0.8430	132.066	8	0.1
303	0.8291	136.570	8	7.4
521	0.8196	140.035	16	14.9
313	0.8160	141.486	16	0.4
440	0.8121	143.091	4	2.2

Table 17: all non-zero reflections of Rutile below 150°.

Anatase (TiO2), I41/amd (141)

hkl	d (Å)	2θ (°)	m	I/I0 (%)
101	3.5165	25.307	8	100.0
103	2.4308	36.951	8	6.1
004	2.3786	37.792	2	20.1
112	2.3323	38.570	8	6.9
200	1.8922	48.043	4	28.0
105	1.7001	53.886	8	17.9
211	1.6663	55.068	16	17.4

hkl	d (Å)	2θ (°)	m	I/I0 (%)
213	1.4932	62.113	16	2.8
204	1.4808	62.689	8	13.5
116	1.3642	68.756	8	5.9
220	1.3380	70.297	4	6.6
107	1.2792	74.052	8	0.5
215	1.2646	75.050	16	10.2
301	1.2506	76.044	8	2.7
008	1.1893	80.737	2	0.4
303	1.1722	82.166	8	0.6
224	1.1662	82.682	8	5.0
312	1.1606	83.166	16	1.8
217	1.0598	93.248	16	0.6
305	1.0514	94.213	8	2.6
321	1.0433	95.178	16	2.9
208	1.0069	99.814	8	1.1
323	0.9965	101.253	16	0.8
316	0.9552	107.489	16	3.9
400	0.9461	109.009	4	2.2
307	0.9246	112.836	8	0.3
325	0.9191	113.884	16	4.2
411	0.9136	114.941	16	2.5
228	0.8889	120.125	8	1.1
413	0.8817	121.774	16	0.8
404	0.8791	122.376	8	3.1
332	0.8767	122.948	8	0.7
420	0.8462	131.083	8	4.9
327	0.8307	136.015	16	0.7
415	0.8267	137.419	16	5.4

Table 18: all non-zero reflections of Anatase below 150°.